Laporan Tugas Besar 1 IF3270 Pembelajaran Mesin

Implementasi Algoritma Feedforward Neural Network

Kelompok 56 KasihTauMama:

Andi Farhan Hidayat	13523128
Andri Nurdianto	13523145
Rafael Marchel D. W.	13523146

Teknik Informatika

Institut Teknologi Bandung

Daftar Isi

1. Deskripsi Persoalan **4**
2. Pembahasan **5**
2.1. Penjelasan Implementasi 5
2.1.1. Deskripsi kelas beserta deskripsi atribut dan methodnya 5
2.1.2. Penjelasan forward propagation 10
2.1.3. Penjelasan backward propagation dan weight update 11
2.2. Hasil Pengujian 12
2.2.1. Pengaruh depth dan width 12
a. Depth = 3 dan Width = 16 12
b. Depth = 3 dan Width = 32 13
c. Depth = 3 dan Width = 64 14
d. Depth = 3 dan Width = 32 15
e. Depth = 4 dan Width = 32 16
f. Depth = 5 dan Width = 32 17
2.2.2. Pengaruh fungsi aktivasi 18
a. Activation Function = Linear 18
b. Activation Function = ReLU 20
c. Activation Function = Sigmoid 22
d. Activation Function = Tanh 24
2.2.3. Pengaruh learning rate 26
a. Learning Rate = 0.1 26
b. Learning Rate = 0.01 29
c. Learning Rate = 0.001 32
2.2.4. Pengaruh inisialisasi bobot 35
a. Zero Init 35
b. Uniform Init (-0.5 - 0.5) 38
c. Normal Init 41
2.2.5. Pengaruh regularisasi 44
a. Tanpa Regularisasi 44
b. L1 Regularisasi (0.001) 47
c. L2 Regularisasi (0.001) 50

2.2.6. Perbandingan dengan library sklearn.....52
Kesimpulan dan Saran..... 53
Kesimpulan..... 53
Saran..... 54
Pembagian tugas tiap anggota kelompok.....56
Referensi..... 57
Lampiran..... 58

1. Deskripsi Persoalan

Feedforward neural network (FFNN) adalah salah satu algoritma pembelajaran mesin jenis ANN (artificial neural network) yang terinspirasi dari struktur syaraf otak manusia. FFNN memiliki dalam menjalani algoritmanya. Terdapat 3 lapis yang berperan dalam perhitungan proses tersebut. Pertama, lapisan input layer sebagai penerima nilai/masukan data. Jumlah neuron dalam input layer sama dengan jumlah banyaknya feature dalam data. Kedua, hidden layer bertugas sebagai lapisan untuk mempelajari pola dari suatu data dengan memanfaatkan bobot (weight) dan bias. Hidden layer bisa memiliki satu atau lebih lapisan di antara input layer dan output layer. Ketiga, output layer sebagai layer terakhir untuk menghasilkan nilai akhir yang dapat diklasifikasi. Jumlah neuron dalam output layer sama dengan jumlah target dari sebuah kelas dalam data.

FFNN memiliki inferensi yang disebut sebagai forward propagation. Forward propagation memanfaatkan nilai weight dan bias yang dimiliki oleh setiap neuron untuk dihitung dan kemudian menjadi input untuk layer selanjutnya. Untuk melatih modelnya, FFNN memiliki backpropagation untuk memperbarui nilai weight dan bias jika hasil memiliki error. Error dihitung menggunakan fungsi loss untuk melihat apakah output yang dihasilkan sama seperti target atau memiliki perbedaan.

Dalam tugas besar ini, FFNN digunakan untuk memprediksi apakah mahasiswa sudah mendapatkan penempatan kerja dan gaji setelah lulus. Data memiliki 10000 instance, 11 feature dan 2 class yang memiliki 2 target yaitu placed dan not placed. Dengan menggunakan FFNN, diharapkan model dapat mengklasifikasi seorang siswa placed atau not placed.

2. Pembahasan

2.1. Penjelasan Implementasi

2.1.1. Deskripsi kelas beserta deskripsi atribut dan methodnya.

Kelas FFNN

Kelas FFNN merupakan kelas utama untuk membuat instance model. Berikut merupakan atribut dan method yang dimiliki kelas FFNN beserta deskripsinya

atribut/method	deskripsi
left_n_neuron	Jumlah neuron layer kiri
right_n_neuron	Jumlah neuron layer kanan
activation_func	Fungsi aktivasi
verbose	Parameter untuk menampilkan detail perhitungan instance
init_method	Pemilihan method inisialisasi untuk weight (zero, uniform, normal)
kwargs	Argumen untuk init_method
forward()	Forward propagation
backward()	Fungsi backward propagation

Kelas Links

Kelas Links merupakan kelas untuk menghubungkan 2 buah layer dalam sebuah instance FFNN. Berikut merupakan atribut dan method yang dimiliki kelas Links beserta deskripsinya

atribut/method	deskripsi
loss	Pemilihan fungsi loss

layers	Inisialisasi layer
activation	Pemilihan fungsi aktivasi
link_verbose	Parameter untuk menampilkan detail perhitungan instance
init_method	Pemilihan method inisialisasi untuk weight (zero, uniform, normal)
kwargs	Argumen untuk init_method
add_link()	Menambah layer pada indeks posisi tertentu
remove_link()	Menghapus layer pada indeks posisi tertentu
fit()	Metode untuk melatih instance
predict()	Metode untuk memprediksi target
save()	Menyimpan model
load()	Me-load model yang sudah disimpan
forward()	Metode untuk melakukan forward propagation
backward()	Metode untuk melakukan backward propagation
update_weight()	Metode untuk memperbarui bobot
apply_reguilarization()	Metode untuk menggunakan regularization
show_weight_distribution()	Metode untuk menampilkan distribusi bobot pada suatu layer
show_dw_distribution()	Metode untuk menampilkan distribusi gradien dan bias pada

	suatu layer

Kelas Activation

Kelas Activation digunakan sebagai kelas abstract dari fungsi aktivasi yang disimpan dalam class. Fungsi aktivasi digunakan untuk menentukan aktif atau tidaknya sebuah neuron dan nilai y. Berikut merupakan atribut dan method yang dimiliki kelas Activation beserta deskripsinya

atribut/method	deskripsi
forward()	Perhitungan fungsi aktivasi untuk forward propagation
backward()	Perhitungan fungsi aktivasi untuk backward propagation

Kelas Linear

Kelas Linear digunakan untuk menyimpan method forward propagation dan backward propagation fungsi aktivasi linear. Berikut merupakan atribut dan method yang dimiliki kelas Linear beserta deskripsinya

atribut/method	deskripsi
forward()	Perhitungan fungsi aktivasi untuk forward propagation
backward()	Perhitungan fungsi aktivasi untuk backward propagation

Kelas ReLu

Kelas ReLu digunakan untuk menyimpan method forward propagation dan backward propagation fungsi aktivasi ReLu. Berikut merupakan atribut dan method yang dimiliki kelas ReLu beserta deskripsinya

atribut/method	deskripsi
forward()	Perhitungan fungsi aktivasi untuk forward propagation
backward()	Perhitungan fungsi aktivasi untuk backward propagation

Kelas Sigmoid

Kelas Sigmoid digunakan untuk menyimpan method forward propagation dan backward propagation fungsi aktivasi sigmoid. Berikut merupakan atribut dan method yang dimiliki kelas Sigmoid beserta deskripsinya

atribut/method	deskripsi
forward()	Perhitungan fungsi aktivasi untuk forward propagation
backward()	Perhitungan fungsi aktivasi untuk backward propagation

Kelas HyperbolicTangent

Kelas HyperbolicTangent digunakan untuk menyimpan method forward propagation dan backward propagation fungsi aktivasi hyperbolic tangent. Berikut merupakan atribut dan method yang dimiliki kelas HyperbolicTangent beserta deskripsinya

atribut/method	deskripsi
forward()	Perhitungan fungsi aktivasi untuk forward propagation
backward()	Perhitungan fungsi aktivasi untuk backward propagation

Kelas Softmax

Kelas Softmax digunakan untuk menyimpan method forward propagation dan backward propagation fungsi aktivasi softmax. Berikut merupakan atribut dan method yang dimiliki kelas Softmax beserta deskripsinya

atribut/method	deskripsi
forward()	Perhitungan fungsi aktivasi untuk forward propagation
backward()	Perhitungan fungsi aktivasi untuk backward propagation

Kelas Loss

Kelas Loss digunakan sebagai kelas abstract dari loss function yang disimpan dalam class. Fungsi aktivasi digunakan untuk menentukan nilai error berdasarkan targetnya. Berikut merupakan atribut dan method yang dimiliki kelas Loss beserta deskripsinya

atribut/method	deskripsi
forward()	Perhitungan error untuk forward propagation
backward()	Perhitungan error untuk backward propagation

Kelas MSE

Kelas MSE digunakan untuk error loss function menggunakan perhitungan MSE. Berikut merupakan atribut dan method yang dimiliki kelas MSE beserta deskripsinya

atribut/method	deskripsi
forward()	Perhitungan error untuk forward

	propagation
backward()	Perhitungan error untuk backward propagation

Kelas BCE

Kelas BCE digunakan untuk error loss function menggunakan perhitungan BCE. Berikut merupakan atribut dan method yang dimiliki kelas BCE beserta deskripsinya

atribut/method	deskripsi
forward()	Perhitungan error untuk forward propagation
backward()	Perhitungan error untuk backward propagation

Kelas CCE

Kelas CCE digunakan untuk error loss function menggunakan perhitungan CCE. Berikut merupakan atribut dan method yang dimiliki kelas CCE beserta deskripsinya

atribut/method	deskripsi
forward()	Perhitungan error untuk forward propagation
backward()	Perhitungan error untuk backward propagation

2.1.2. Penjelasan forward propagation.

Forward propagation adalah inferensi dari FFNN untuk menentukan kelas target. Pada proses ini data dilatih berdasarkan bobot dan bias pada

sebuah layer. Perhitungan dalam forward propagation akan mempelajari pola dan hasil target akhir.

Pada proses pertama, data awal dimasukan ke input layer. Setiap feature dalam dataset sesuai dengan neuron di lapisan ini. Pada tahap ini, model membaca semua informasi yang dibutuhkan. Sebelum masuk ke jaringan, data dinormalisasi atau distandarisasi agar perhitungan menjadi lebih baik.

Kedua, data yang telah ada pada input layer akan diteruskan ke hidden layer lalu hasil dihitung dengan formula z = w . x + b. Kemudian menerapkan fungsi aktivasi untuk dimasukan pada layer selanjutnya.

Terakhir, output layer digunakan untuk memprediksi hasil dari model. Sama seperti hidden layer, pada proses ini hasil perhitungan menggunakan fungsi aktivasi untuk menghasilkan output yang lebih bermakna.

2.1.3. Penjelasan backward propagation dan weight update.

Backward propagation adalah proses model dilatih. Pada proses ini bobot dan bias instance dari model diperbarui untuk meminimalkan error. Dalam proses ini, model dilatih dari bagian output layer dengan mencari error pada setiap layernya. Backward propagation dikombinasikan dengan metode optimisasi seperti gradient descent. Backward propagation meminimalkan error pada setiap iterasinya.

Tahap yang dilakukan sebelum proses backward propagation adalah memasukan nilai target pada output layer. Lalu dihitung error menggunakan loss function untuk melihat seberapa jauh prediksi target dengan nilai asli.

Perhitungan gradien dari sebuah unit model dirumuskan dengan menggunakan chain rule sebagai berikut.

$$\frac{\partial E_d}{\partial w_{ji}} = \frac{\partial E_d}{\partial o_j} \cdot \frac{\partial o_j}{\partial net_j} \cdot \frac{\partial net_j}{\partial w_{ji}}$$

Lalu untuk tahapan dalam perhitungan backward propagation dan memperbarui bobot dapat dilihat sebagai berikut.

1. Menghitung gradien menggunakan formula

$\Delta w_{ij}=\eta\times\delta_j\times O_j$

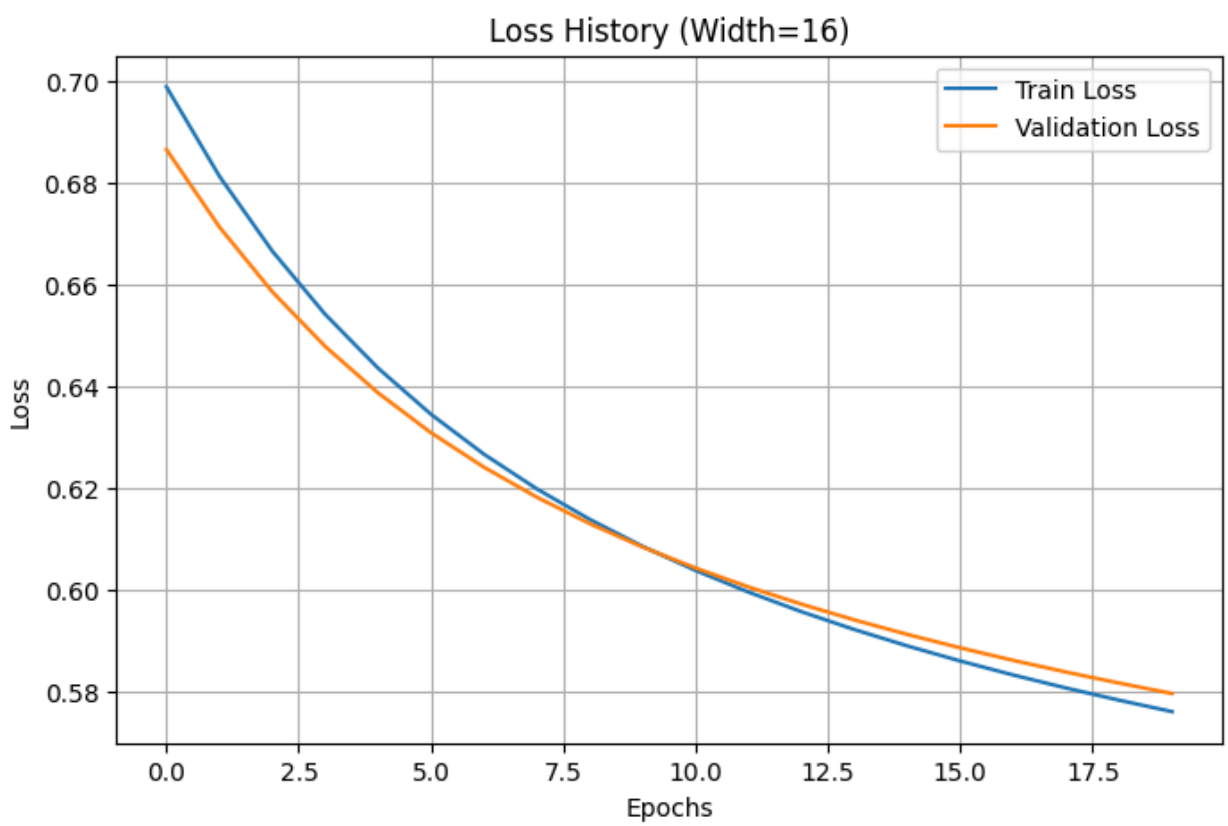
2. Hitung error dari unit output (formula bisa berubah sesuai dengan fungsi aktivasi)
   $\delta_j=y_j(1-y_j)(y_{target}-y_j)$
3. Hitung error dari unit hidden
   $\delta_j=y_j(1-y_j)(w_{ij}\times\delta_j)$
4. Perbarui bobot baru
   $w_{ij} <- w_{ij} + \Delta w_{ij}$

Proses backward propagation terus diulang sampai iterasi terakhir dan diharapkan error dapat berkurang karena perubahan nilai bobot oleh

2.2. Hasil Pengujian

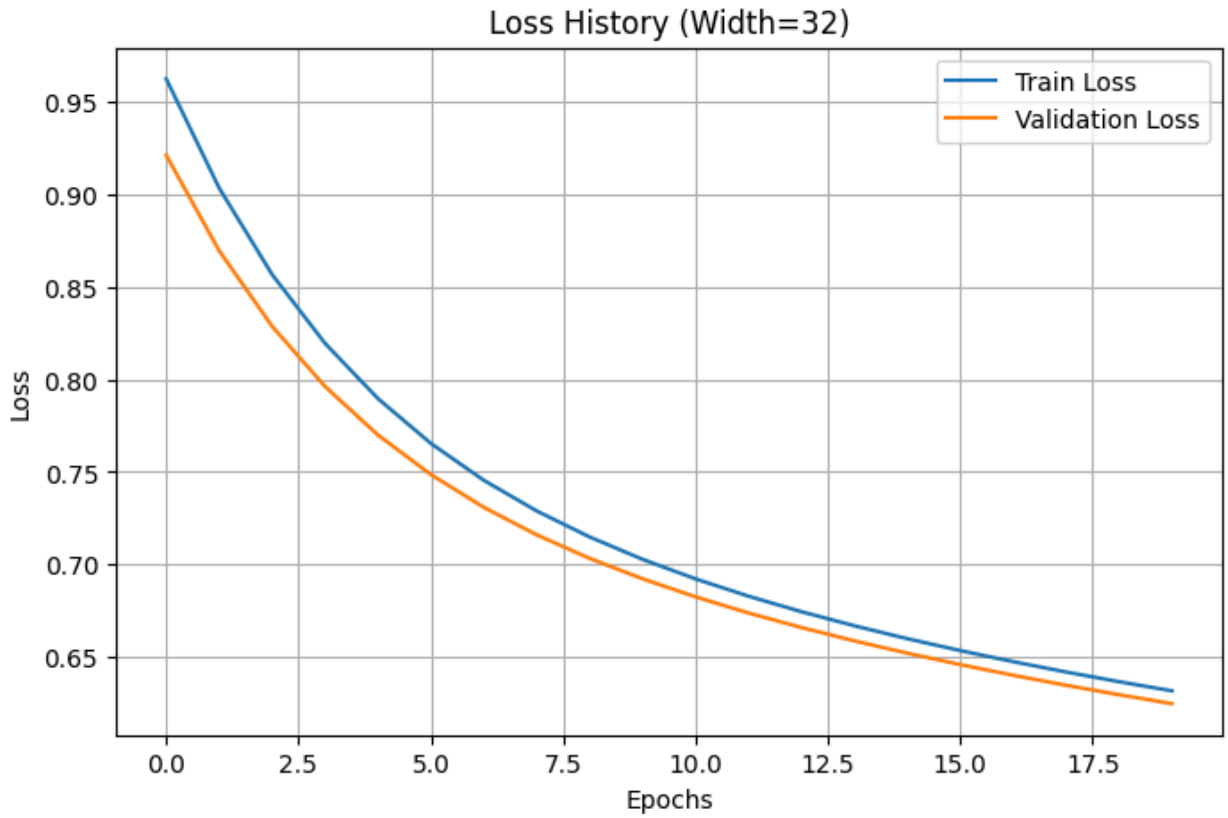
2.2.1. Pengaruh depth dan width

a. Depth = 3 dan Width = 16



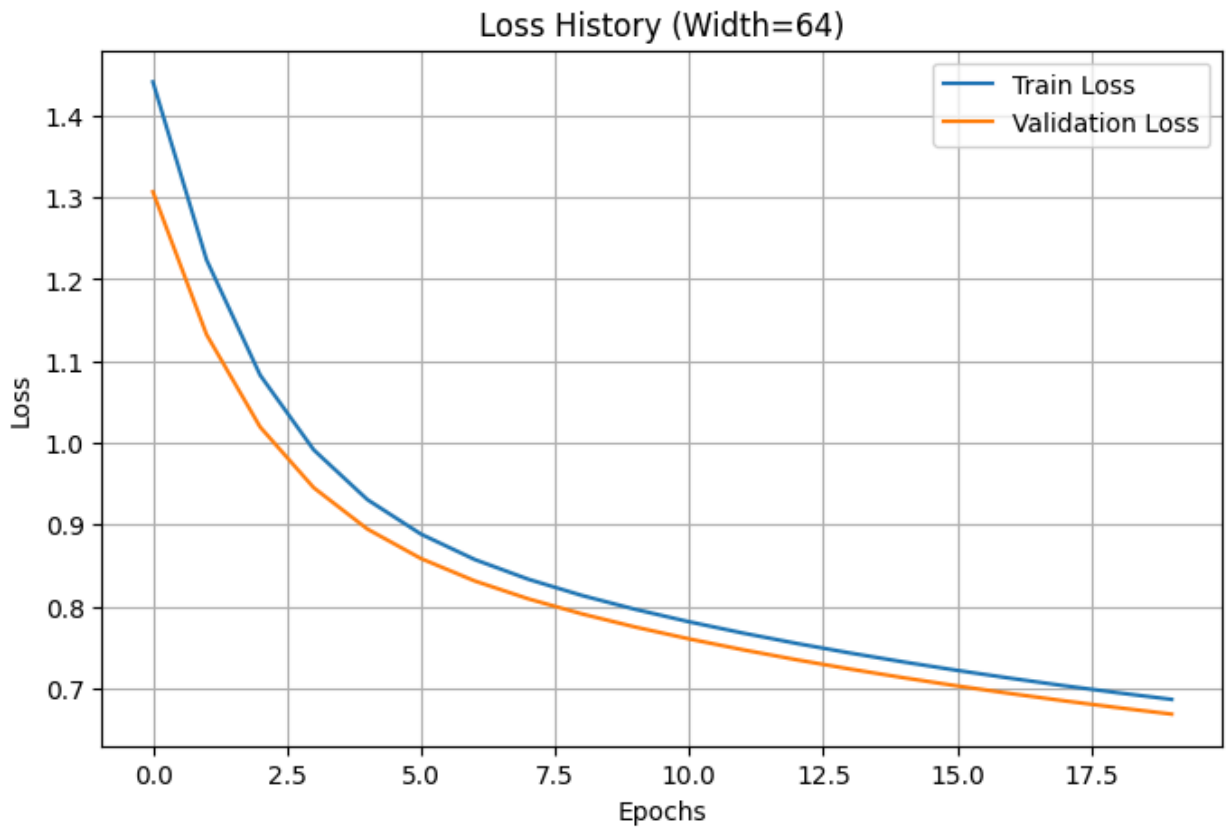
Grafik *Loss History* menunjukkan kurva *Train Loss* (biru) dan *Validation Loss* (oranye) turun secara mulus hampir bersamaan dari angka kisaran 0.70 hingga ke batas 0.58 dalam 20 epoch. Posisi kurva Validation Loss yang konsisten berada sedikit di bawah Train Loss menandakan bahwa model mempelajari data dengan baik tanpa indikasi overfitting, meskipun ujung grafiknya masih sedikit menukik ke bawah yang mengisyaratkan epoch pelatihannya belum sepenuhnya mencapai garis datar konvergensi konstan.

b. Depth = 3 dan Width = 32



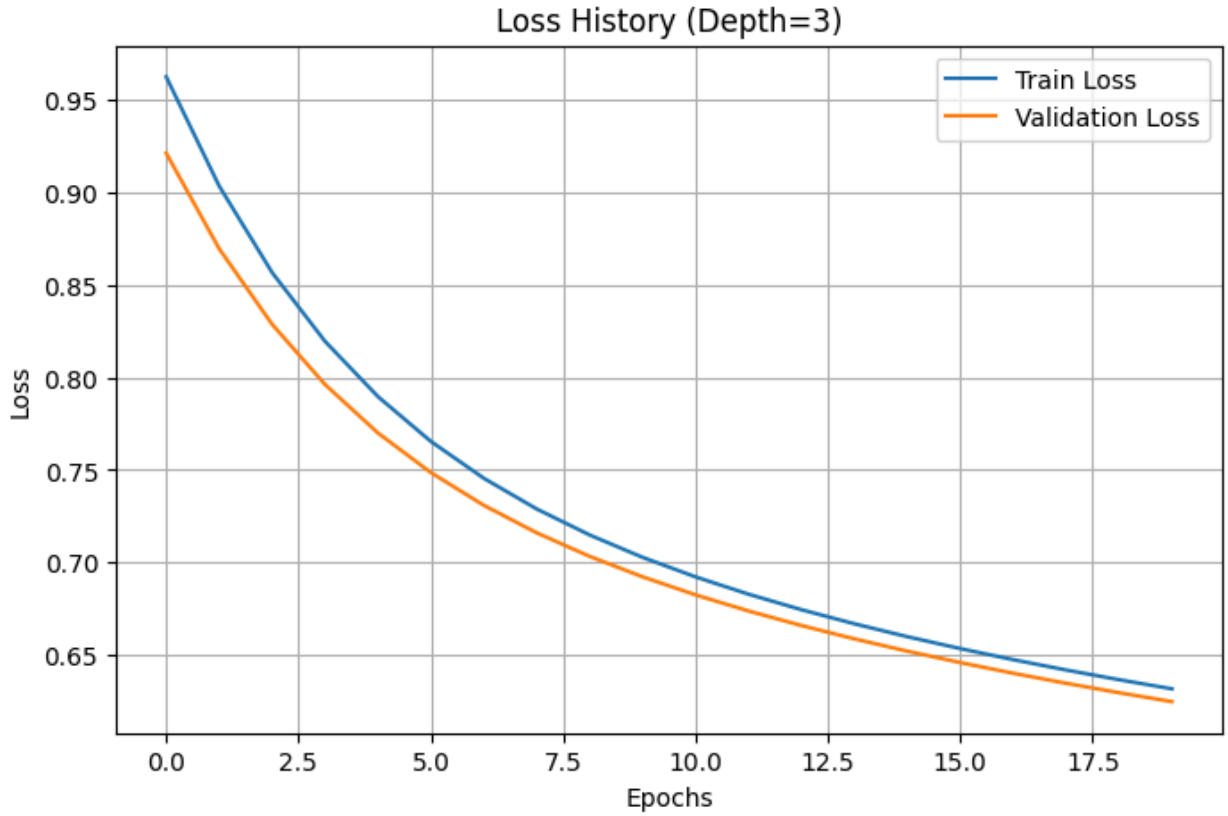
Dibandingkan dengan ukuran sebelumnya, Loss dimulai dari nilai yang lebih tinggi (sekitar 0.95), namun laju penurunannya (slope) di lima epoch pertama terlihat lebih curam. Hasil akhirnya berada di loss margin kisaran 0.63 (kurva biru) dan 0.62 (kurva oranye); konvergensi pembelajarannya masih normal dan terhindar dari bias berlebih.

c. Depth = 3 dan Width = 64



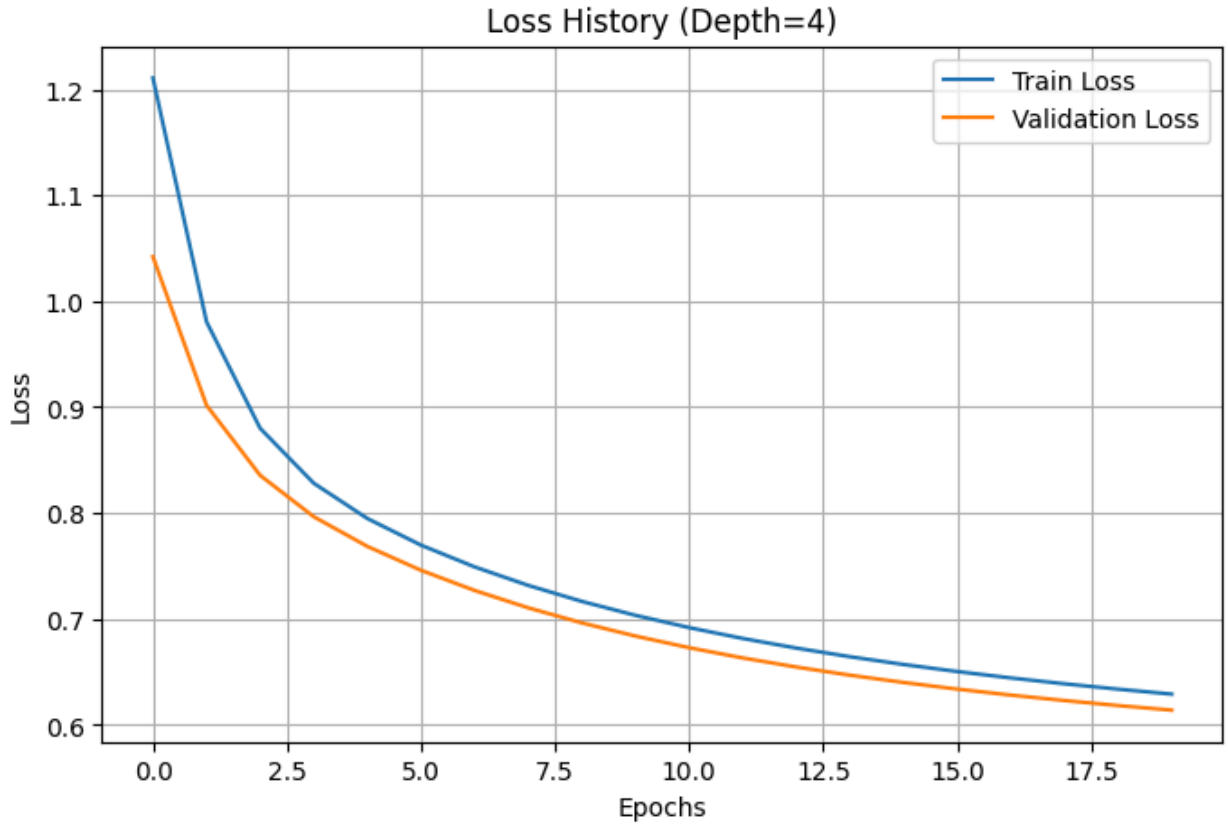
Dengan penambahan lebar neuron menjadi 64, titik awal Train Loss melonjak drastis ke angka 1.4 dan Validation Loss ke 1.3. Meskipun grafiknya menunjukkan penurunan curam asimtotis yang baik dan akhirnya landai di sekitar nilai 0.68 di epoch ke-20, nilai error akhir ini secara visual faktanya lebih tinggi ketimbang pengujian Width berukuran 16 atau 32 sebelumnya. Ini membuktikan untuk struktur 3 layer, pelebaran perseptron (width) justru menunda model untuk mencapai loss paling minimum dalam batas 20 iterasi yang sama.

d. Depth = 3 dan Width = 32



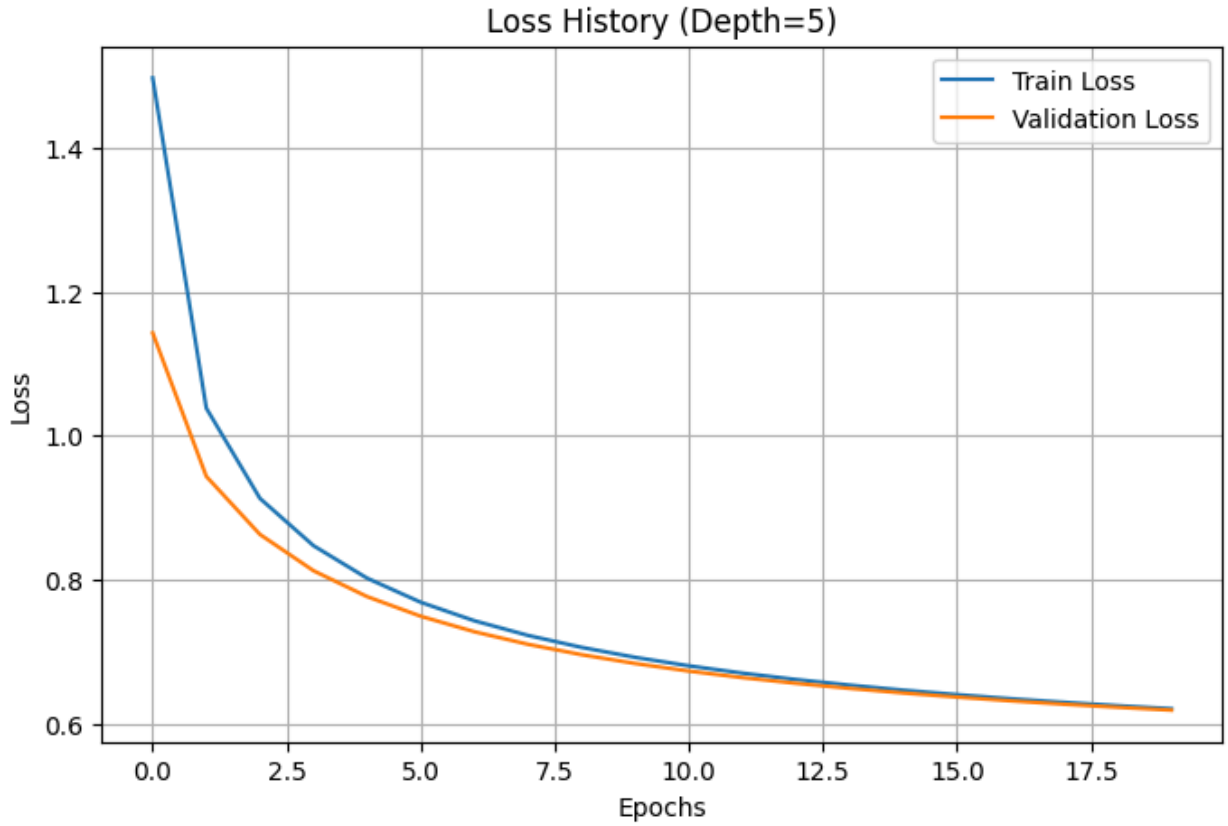
Grafik Loss History (Depth=3) pada bagian ini menampilkan turunan Train Loss yang identik seperti turunan kurva eksponensial halus dari kisaran 0.96 ke area 0.63. Karena kedua garis bergerak rapat menyusut tanpa adanya pantulan pisah ke atas, model mampu menyelaraskan pelajaran di data uji tanpa mengingat fitur dari data pelatihannya.

e. Depth = 4 dan Width = 32



Penambahan struktur ketebalan menjadi 4 layer (Depth=4) membuat inisiasi loss awal sedikit lebih tinggi (1.2 untuk Train dan 1.05 untuk Val), namun model dengan sangat laju mengejar penurunannya menuju angka 0.61 dan 0.63. Garis oranye semakin rapat dan menempel di bawah garis biru pada akhir epoch, menandakan model menggeneralisasi dengan mulus.

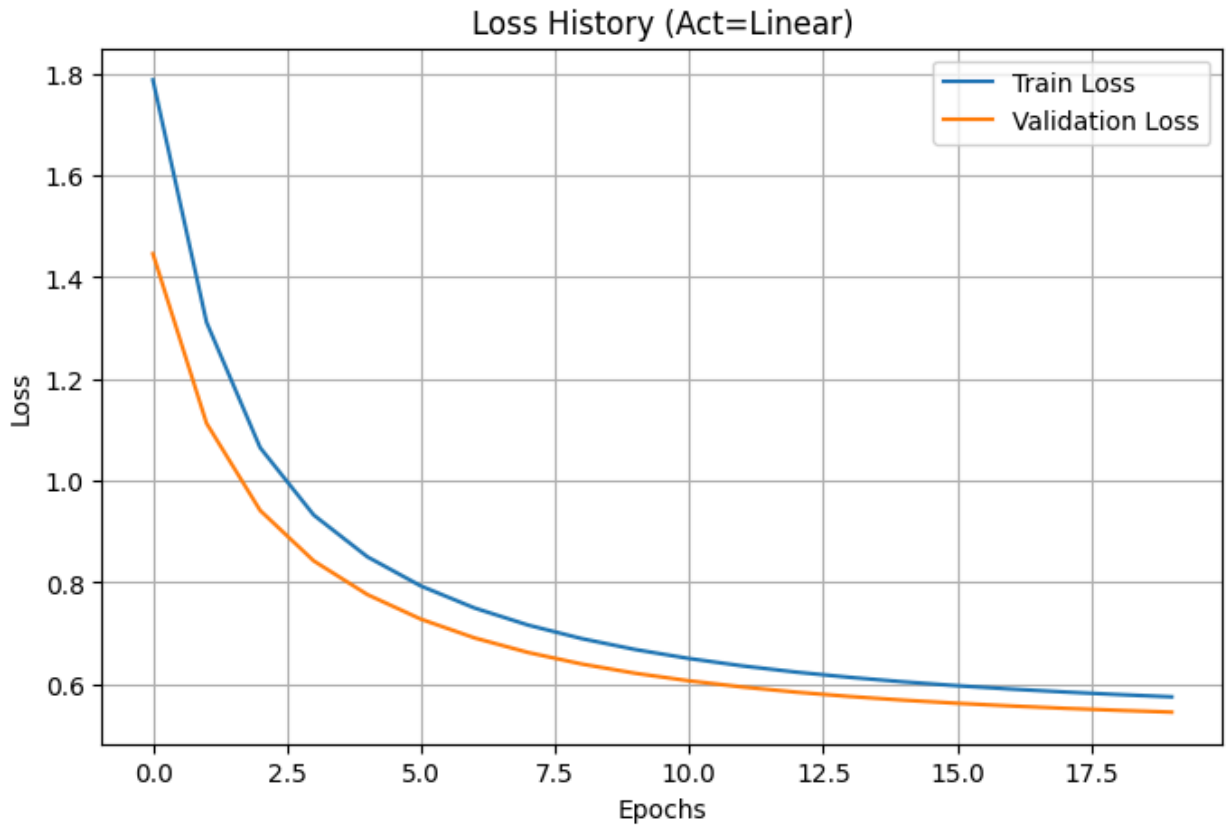
f. Depth = 5 dan Width = 32

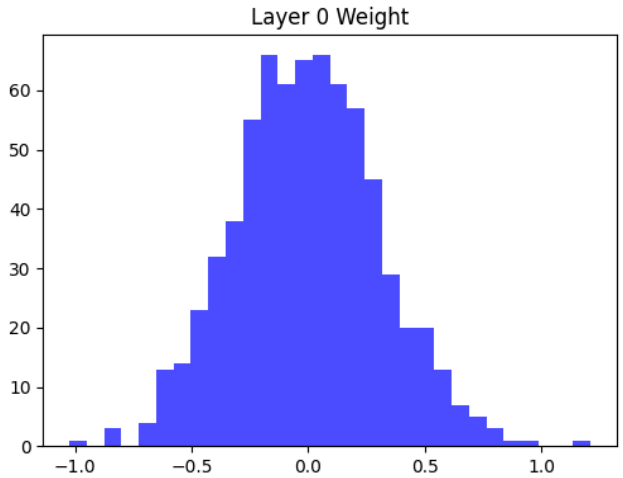


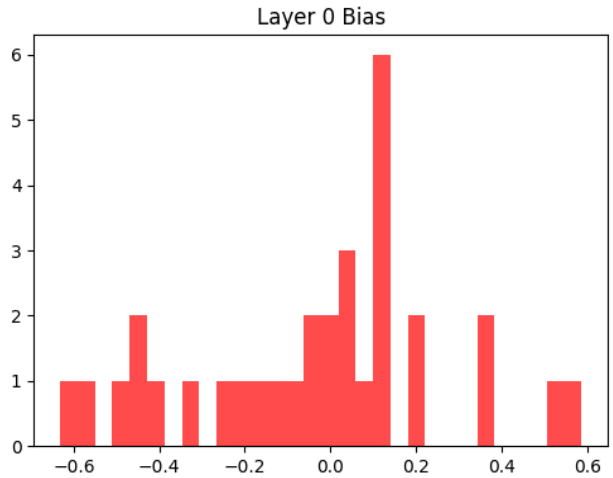
Pembacaan visual mengindikasikan bahwa pada pengujian 5 kedalaman lapis ini, kurva sempat terjun bebas dari loss maksimum 1.5 menyusut sampai garis pelandaian konvergensinya di ambang 0.62. Keunikan utama dari gambar pengujian depth tertinggi ini adalah bertemunya secara sangat akurat (bertumpuknya) ujung garis ujung Train Loss dan Validation Loss tepat pada angka kisaran 0.62 di iterasi pungkasan.

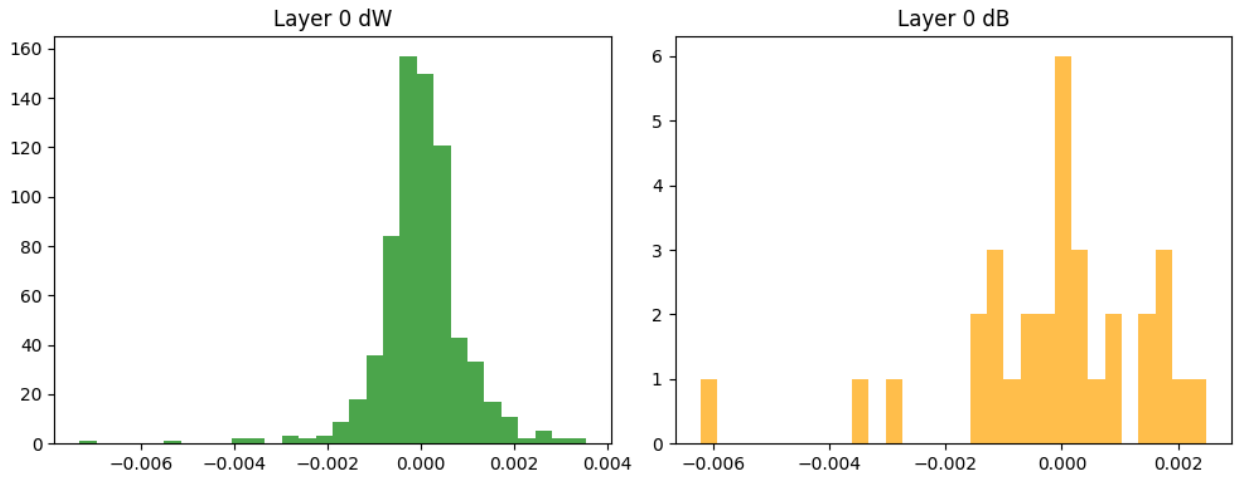
2.2.2. Pengaruh fungsi aktivasi

a. Activation Function = Linear



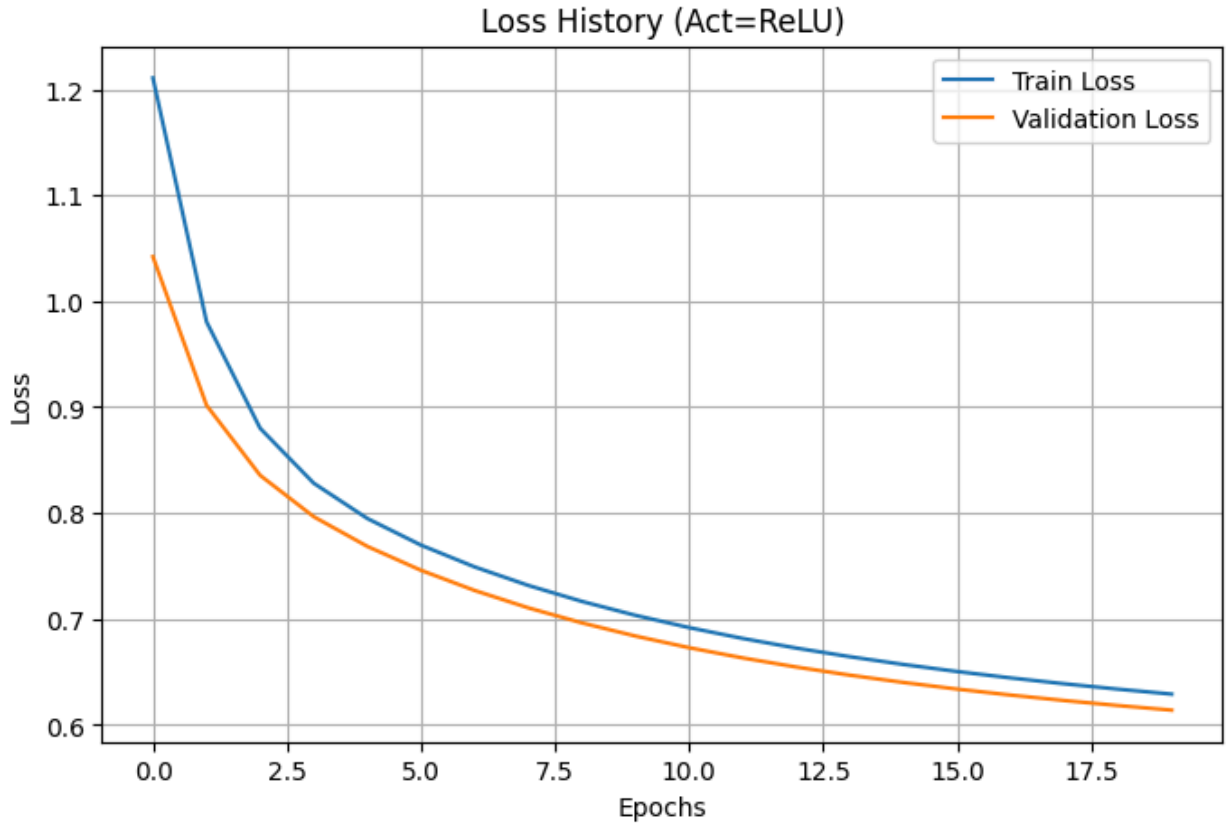


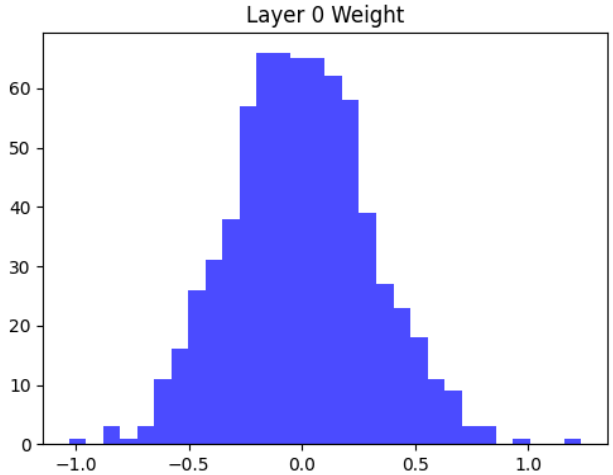


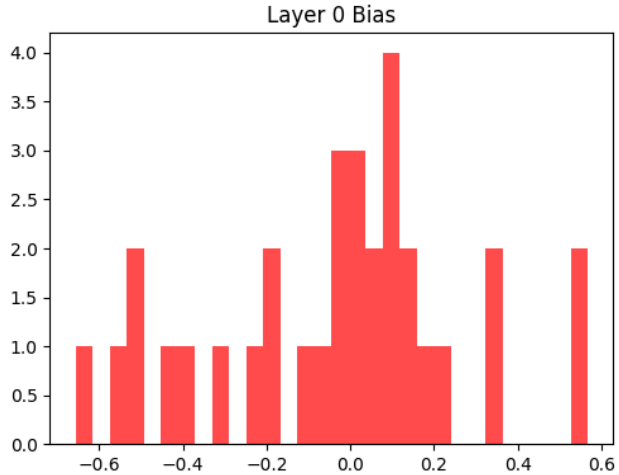


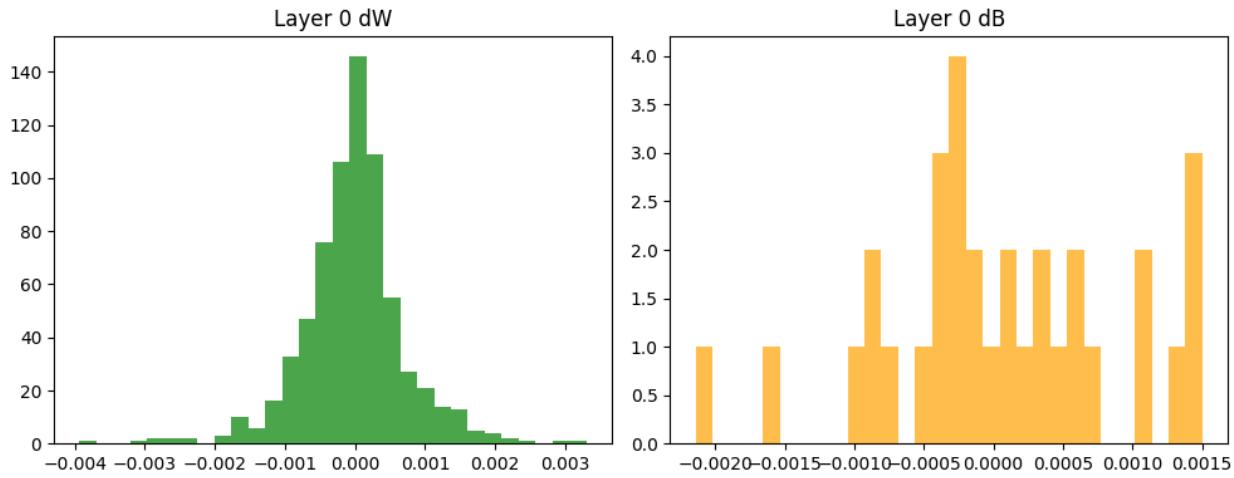
Pergerakan laju error aktivasi Linear bergerak turun dari angka batas 1.8 menyentuh batas konvergensi minimum di 0.55. Di samping itu, dua histogram Layer 0 Weight memperlihatkan bentuk persebaran kurva lonceng simetris yang membentang normal dari rentang -1.0 ke 1.0. Pada plot Layer 0 dW, dominasi pemusatan histogram terpadat di titik nol (0.000) terukur stabil, dengan penyebaran yang sangat ketat meruncing merepresentasikan konvergensi gradien yang solid.

b. Activation Function = ReLU



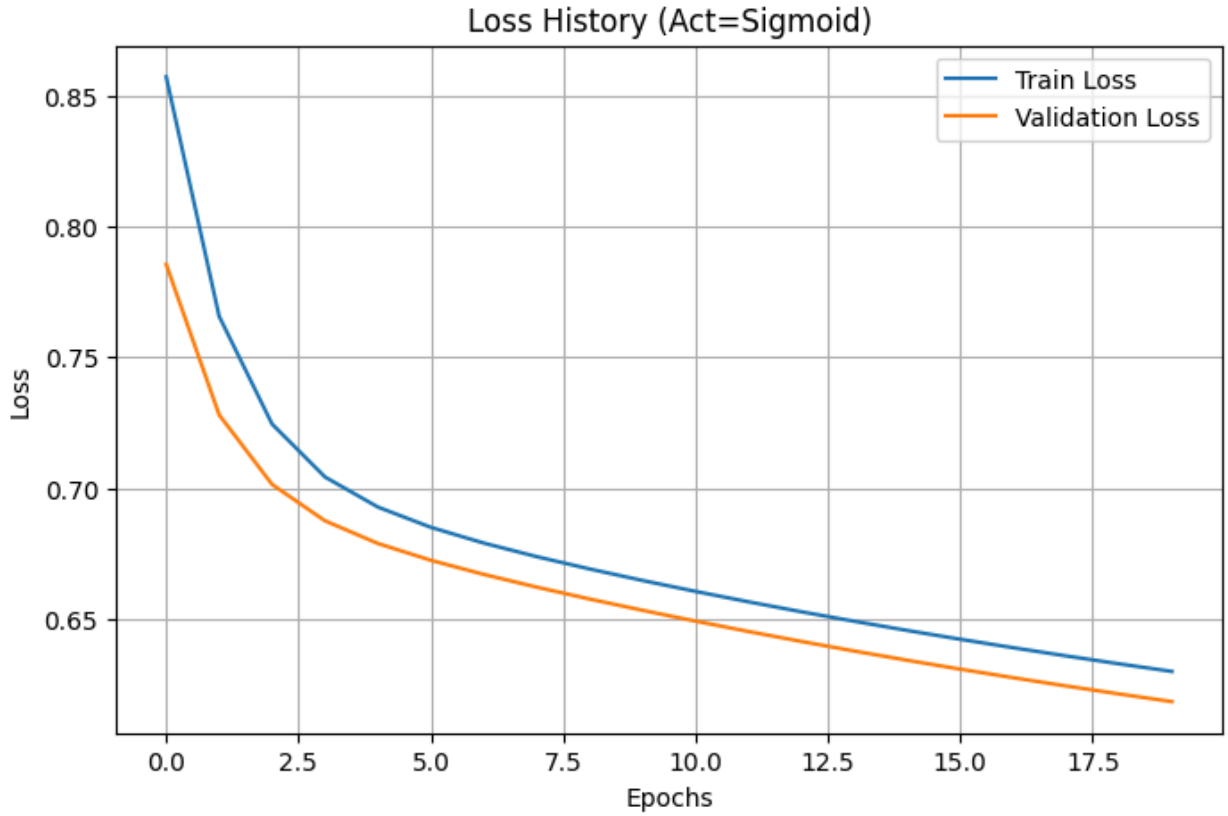


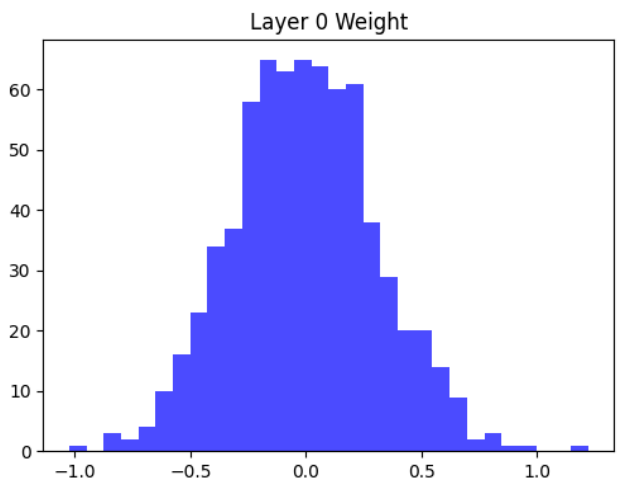


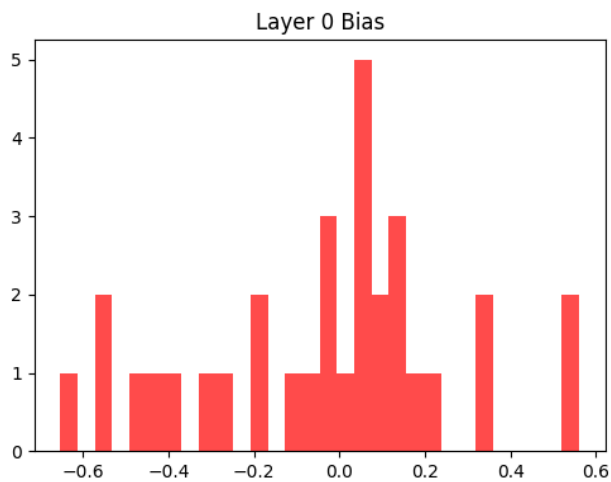


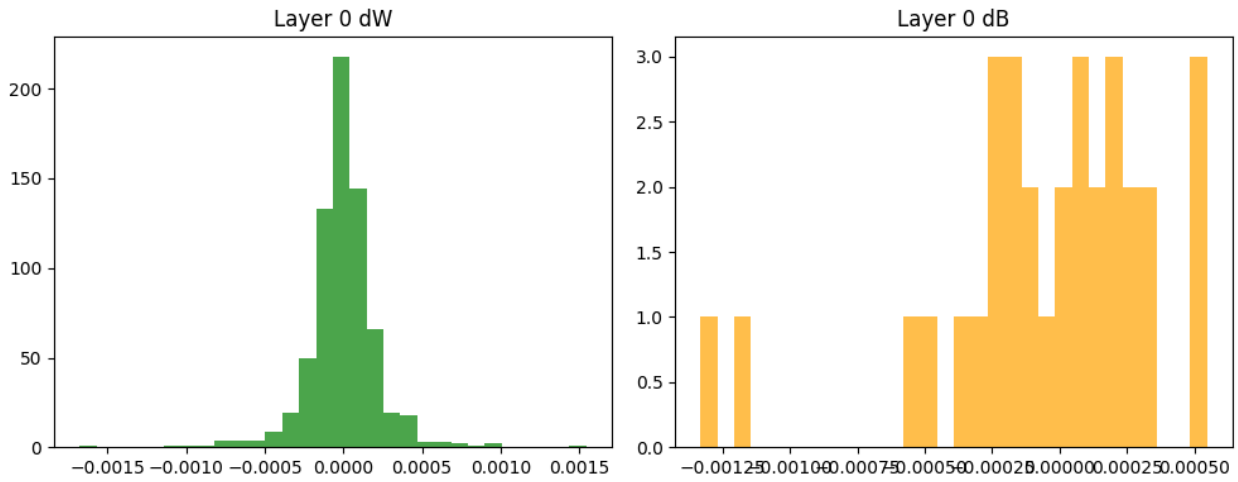
Plotting Train/Val Loss History menyempit curam dari atas 1.2 hingga terhenti di 0.63 pada garis ekornya. Serupa dengan Linear, visualisasi Layer 0 Weight mendistribusikan blok-blok histogram biru yang memusat wajar (bell-shaped) di tengah. Persebaran visual bar hijau untuk gradien dW menunjukkan peak tinggi di angka indeks sumbu-X 0.000 dengan rentangan nilai sebaran ekstrim antara -0.004 sampai 0.003.

c. Activation Function = Sigmoid



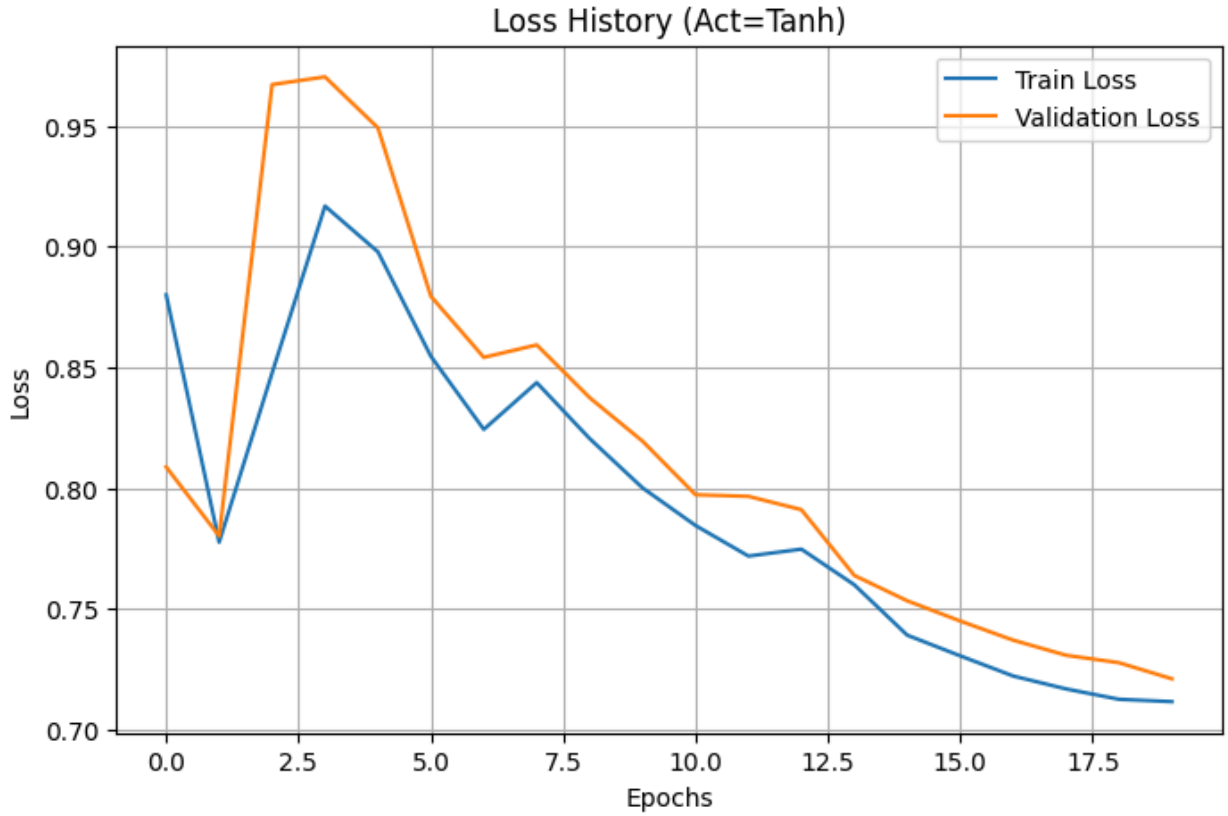


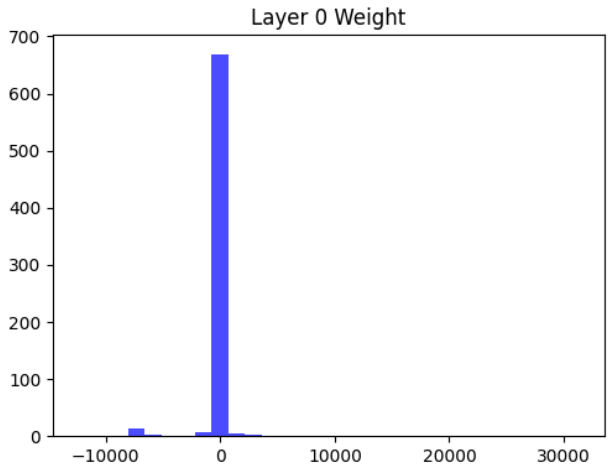


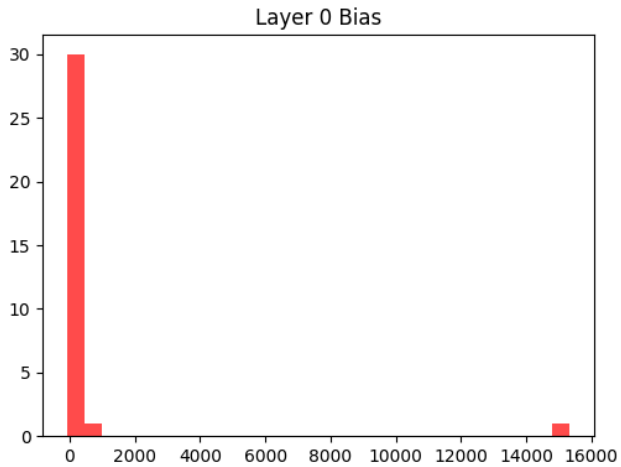


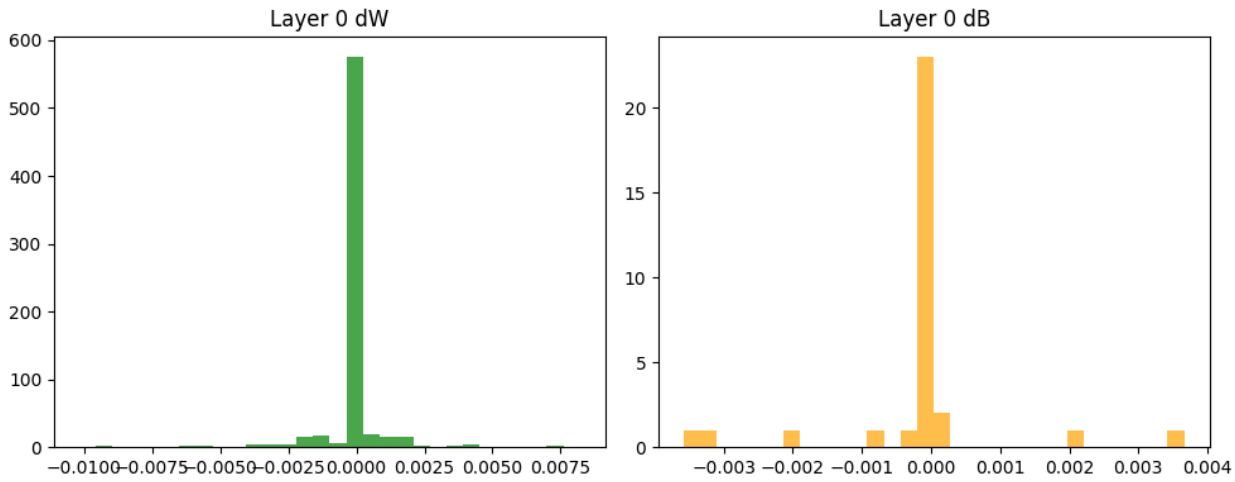
Aktivasi Sigmoid mencatatkan perputaran grafik loss yang mendatar lebih lamban, memulai evaluasinya dari nilai rendah 0.85 dan tertekan santai hingga batas 0.63 akhir. Gambar sebaran kurva birunya tidak sedrastis bentuk histogram yang lain tetapi mengumpul seragam dengan penyebaran nilai bobot yang tak melewati sumbu -1.0 maupun 1.0, menjadikannya fungsi dengan karakteristik penyesuaian bobot paling homogen.

d. Activation Function = Tanh





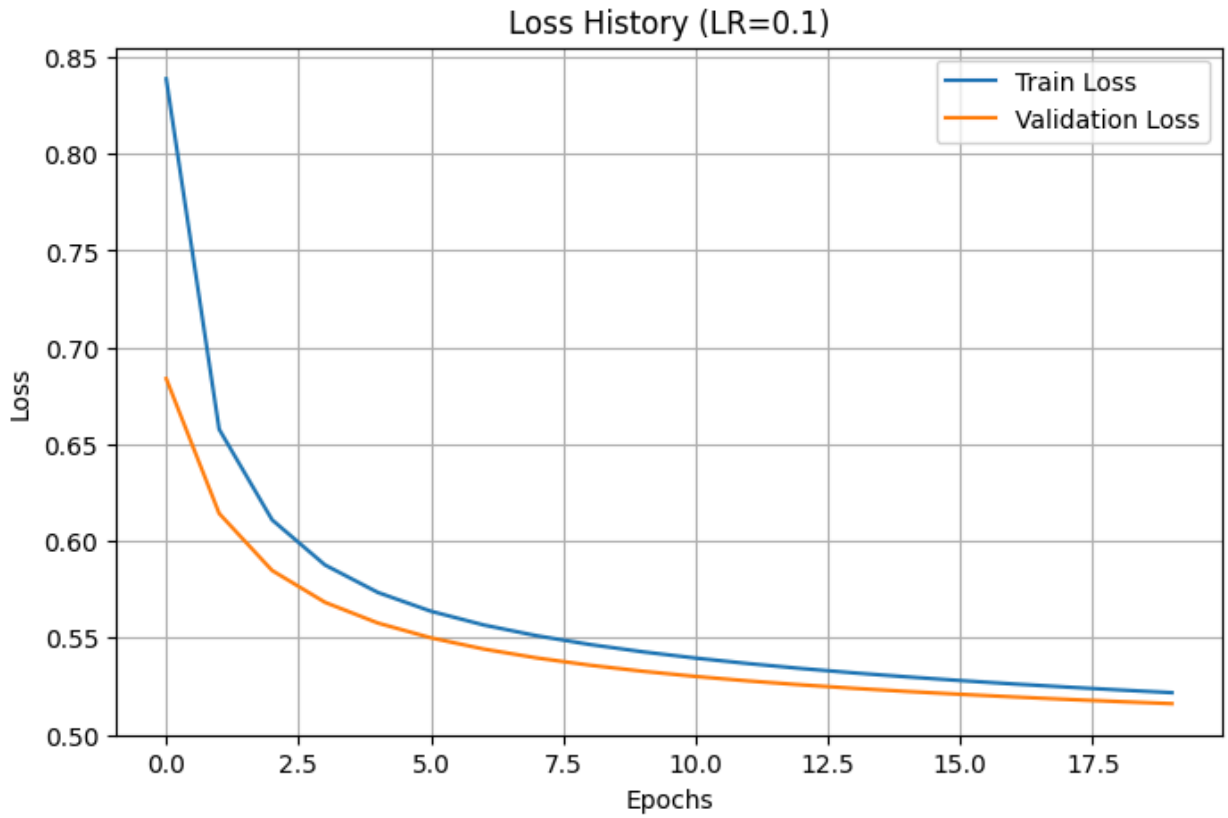


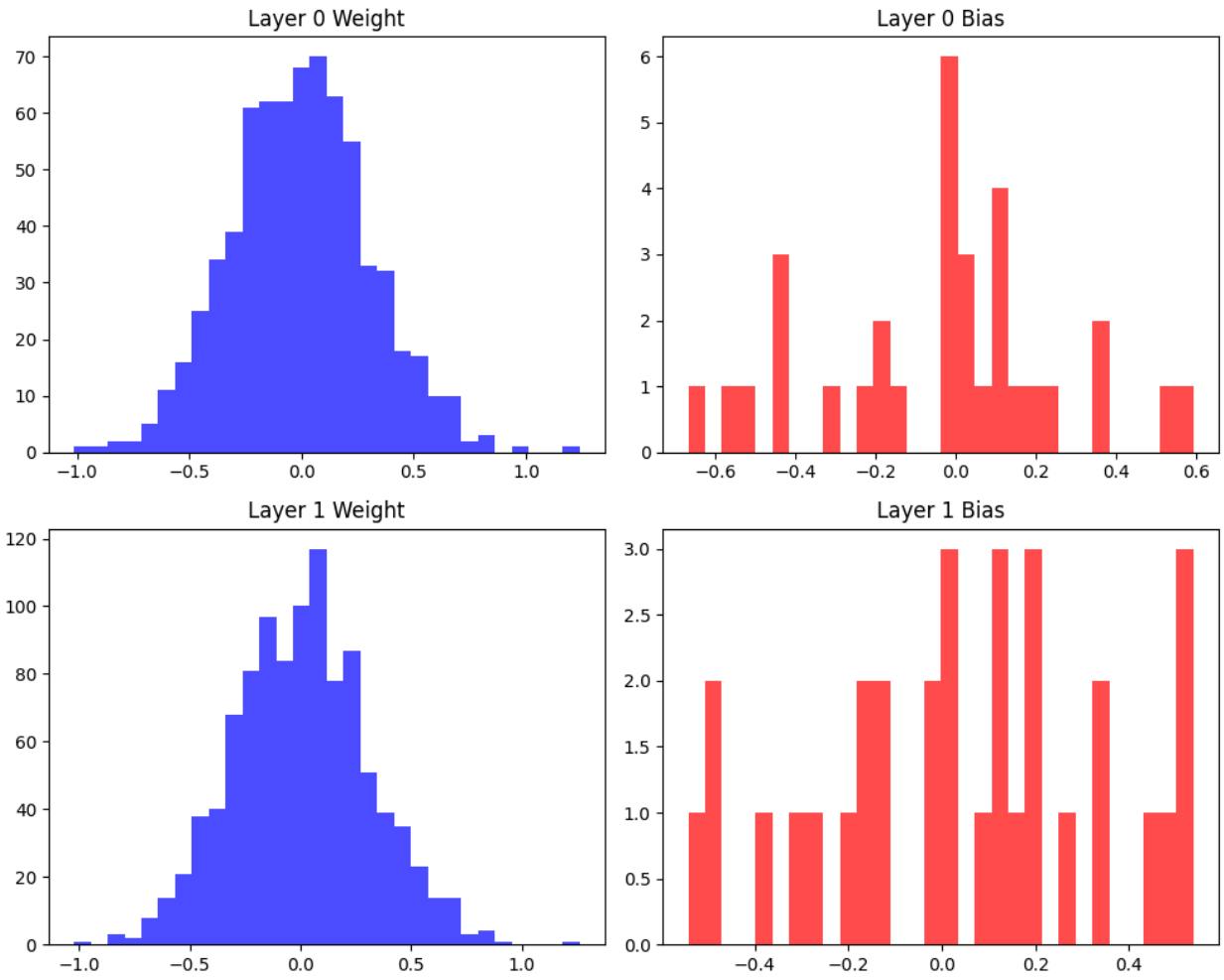


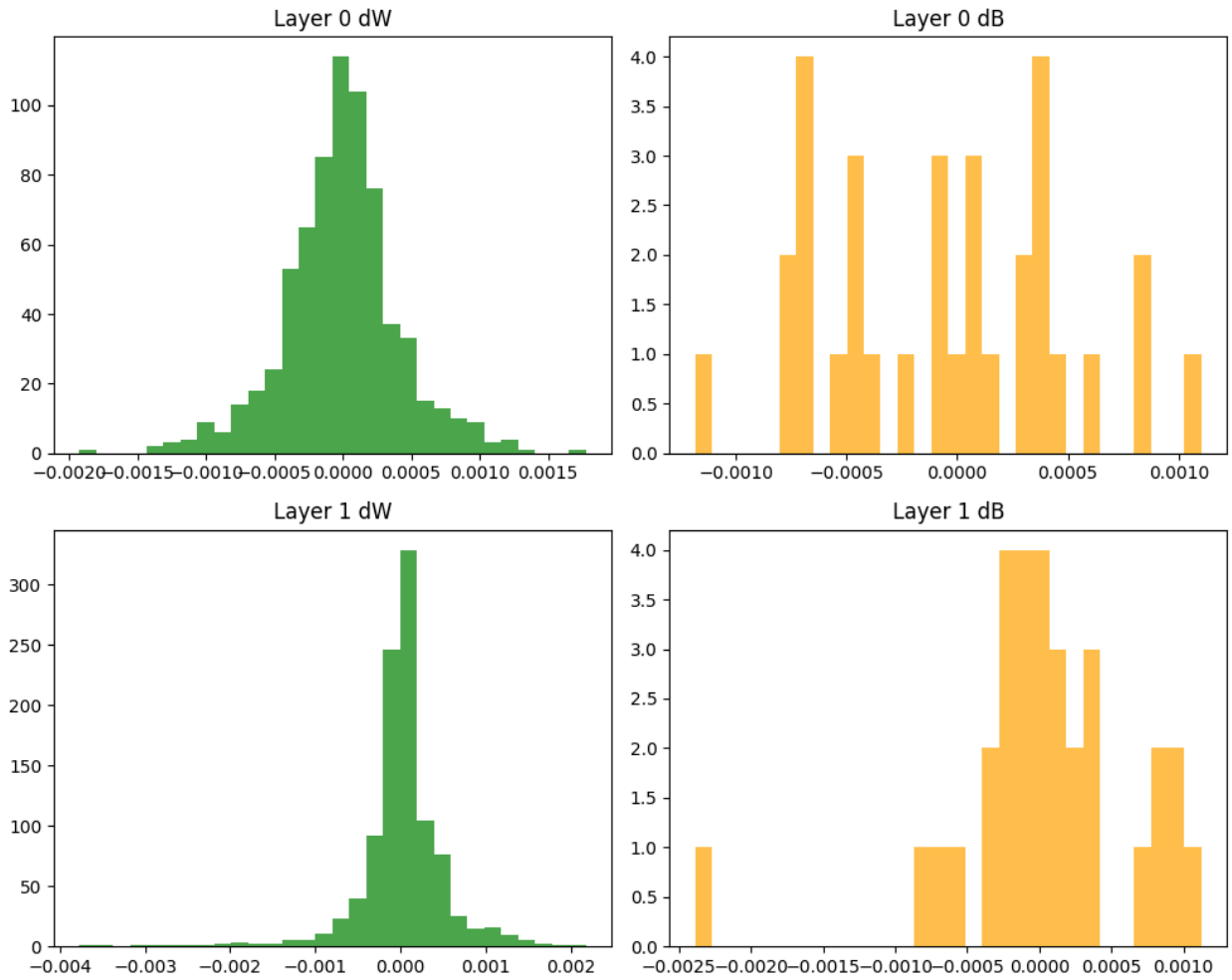
Visualisasi grafik aktivasi Tanh memperlihatkan fenomena spiking yang tajam (lonjakan ketidakstabilan pada graf garis log). Garis yang semula bernilai sekitar 0.88 mendadak turun di epoch 1, lalu melenting drastis rusak melempar naik ke puncak 0.96 di epoch 3 sebelum model akhirnya sembuh dan menurun asimtotis kembali ke 0.72 di akhir percobaan. Hal ini diperkuat oleh wujud bar grafik histogramnya (weight dan dW) yang sama sekali menyimpang dari persebaran lonceng; nilainya mengerucut sangat memuncak hanya murni di titik simetrik nol tengahnya (distribusi sangat mampat nyaris berbentuk satu tiang lurus tinggi).

2.2.3. Pengaruh learning rate

a. Learning Rate = 0.1

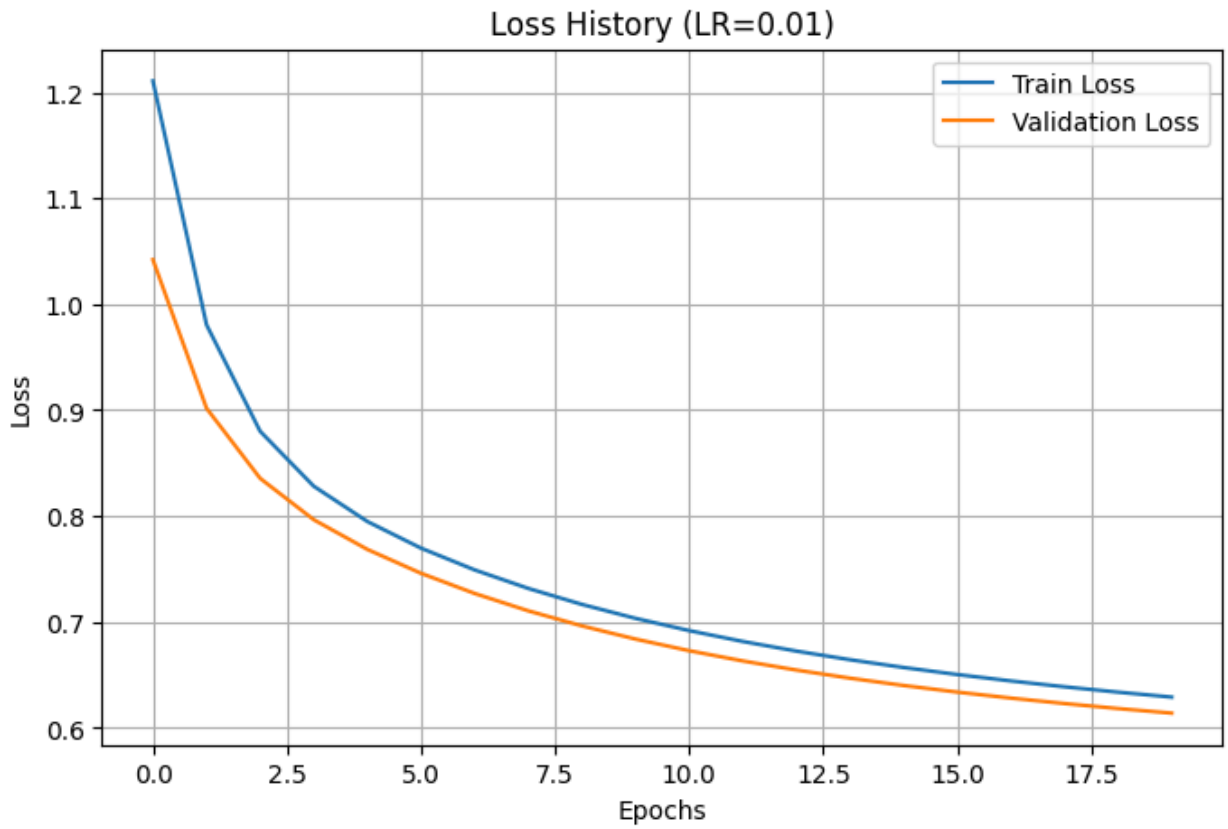


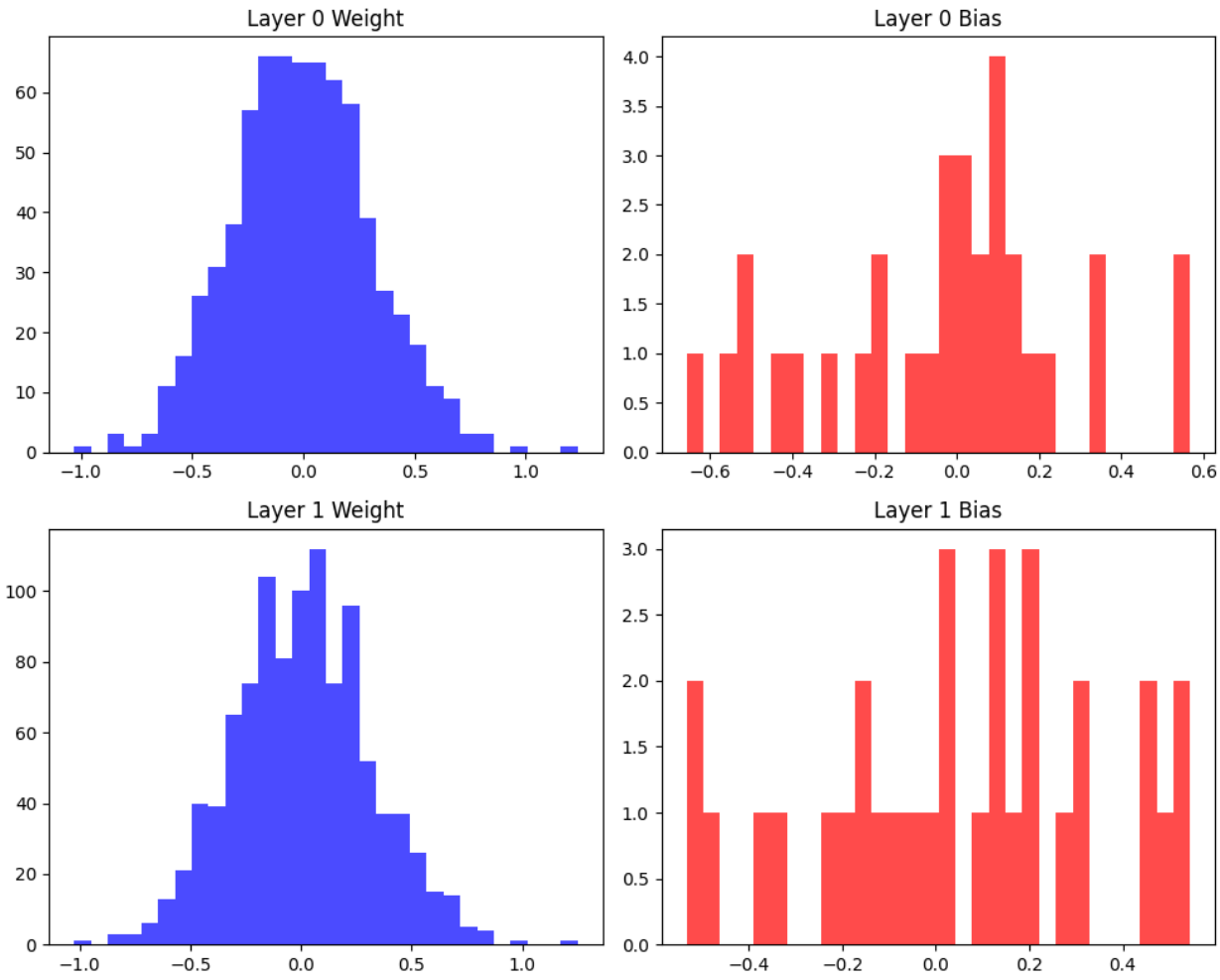

Layer 0 Weight
Layer 0 Bias
Layer 1 Weight
Layer 1 Bias

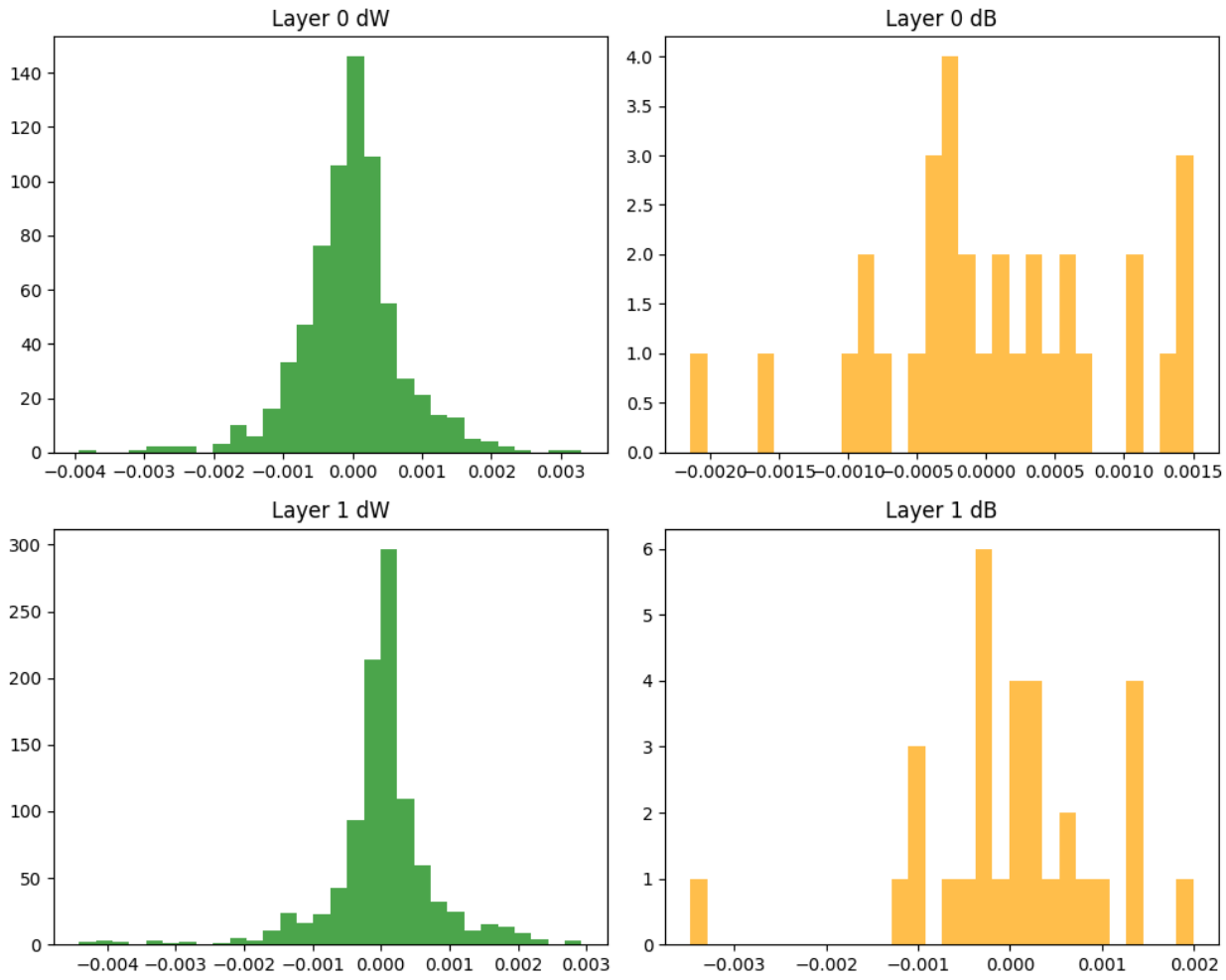


Pada laju LR tertinggi ini, grafik penurunan merosot sangat gesit secara eksponensial di 5 epoch awal dari arah 0.85 dan mendatar stabil secara prematur menyentuh dataran rendah 0.52 (posisi konvergen paripurna). Diagram bar biru dan hijau di bawahnya berskala merata secara bimodal padat yang menjustifikasi kebenaran pembaruan bobot dalam jangkauan nilai sumbu normal.

b. Learning Rate = 0.01

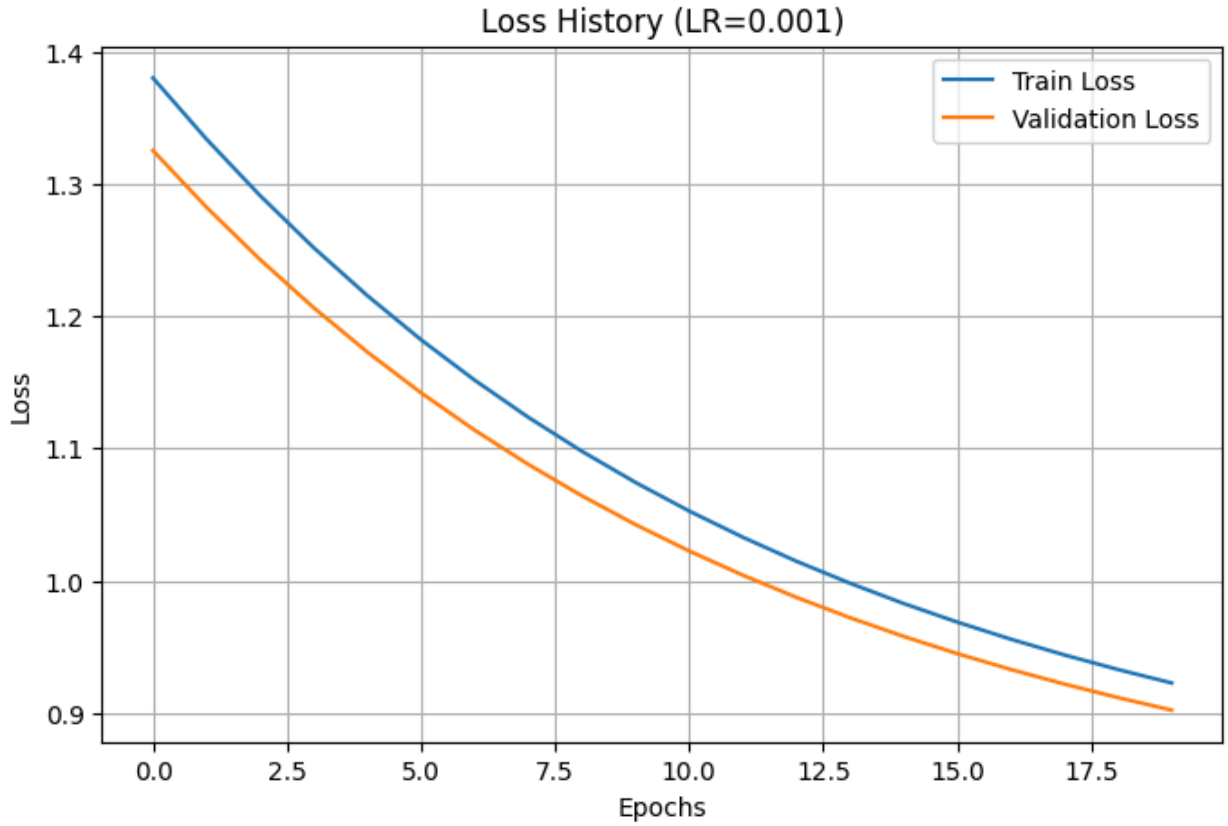


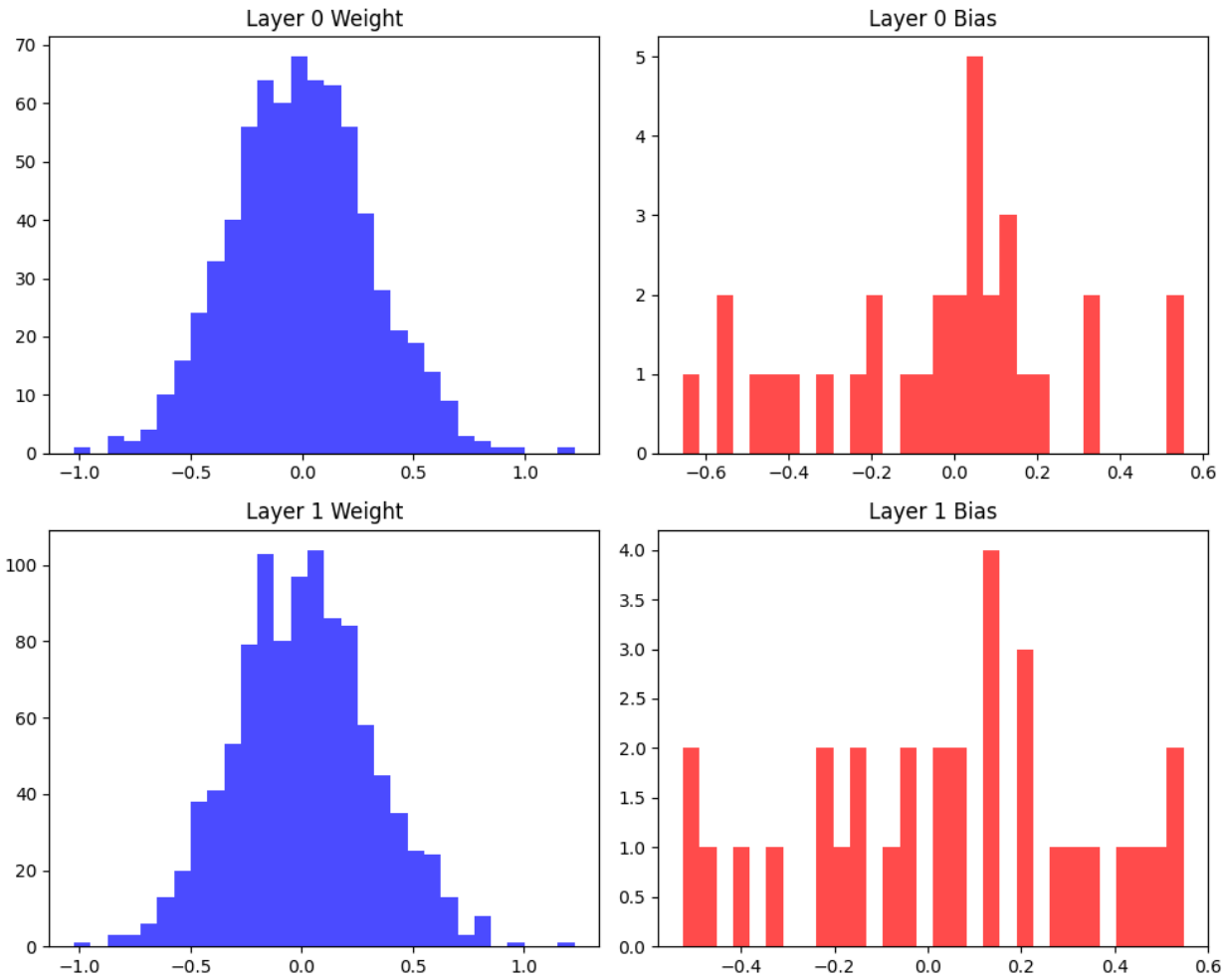

Layer 0 Weight
Layer 0 Bias
Layer 1 Weight
Layer 1 Bias

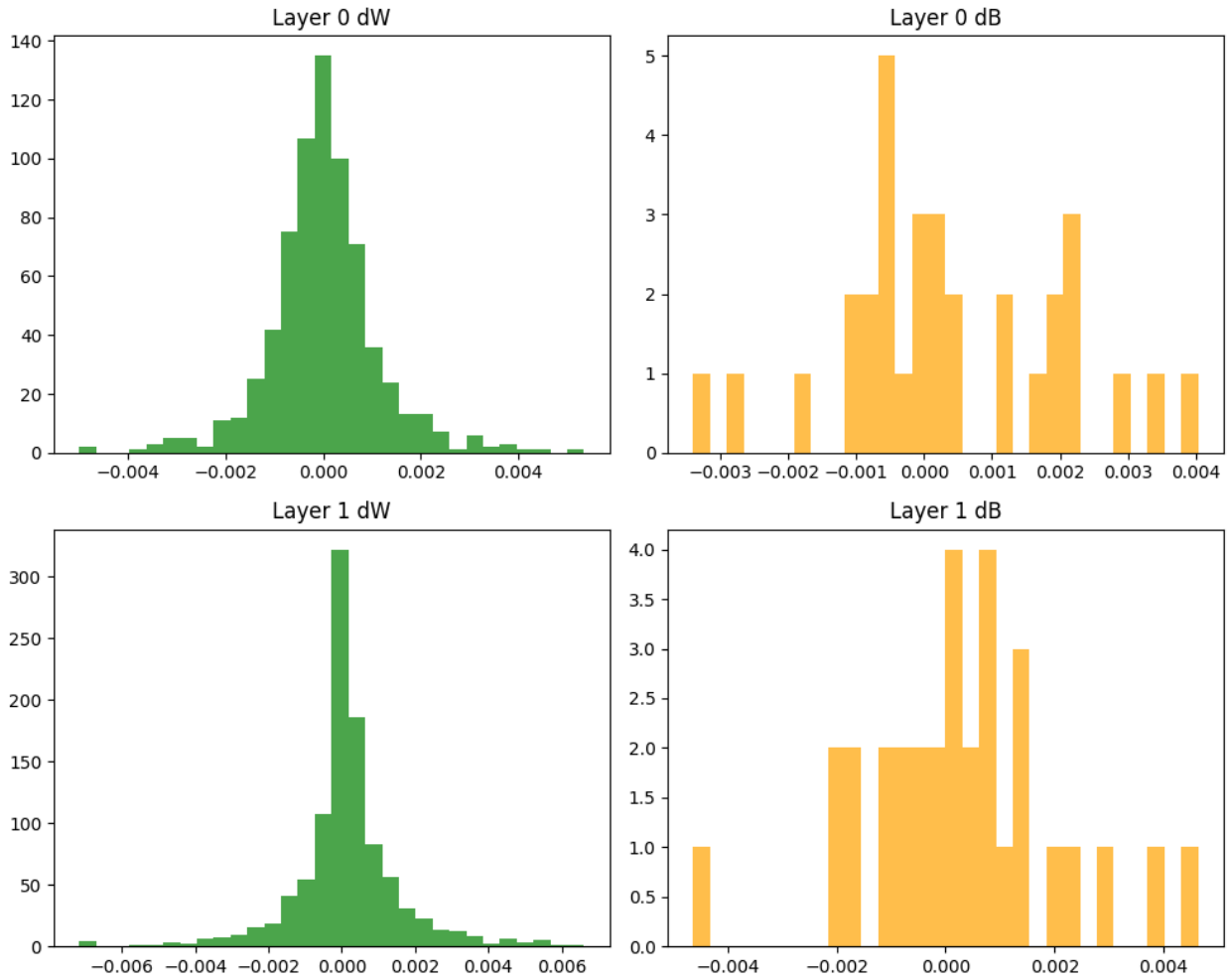


Turunan LR sepuluh kali lipat melahirkan grafik kurva loss rate yang menyusut jauh lebih gradual atau proporsional tanpa ada hentakan berderet dari sumbu 1.2 ke 0.62. Histogram untuk persebaran weight Layer 0 maupun Layer 1 mendemonstrasikan sebaran gundur gaussian mulus yang ideal melebar dari kiri ke kanan.

c. Learning Rate = 0.001



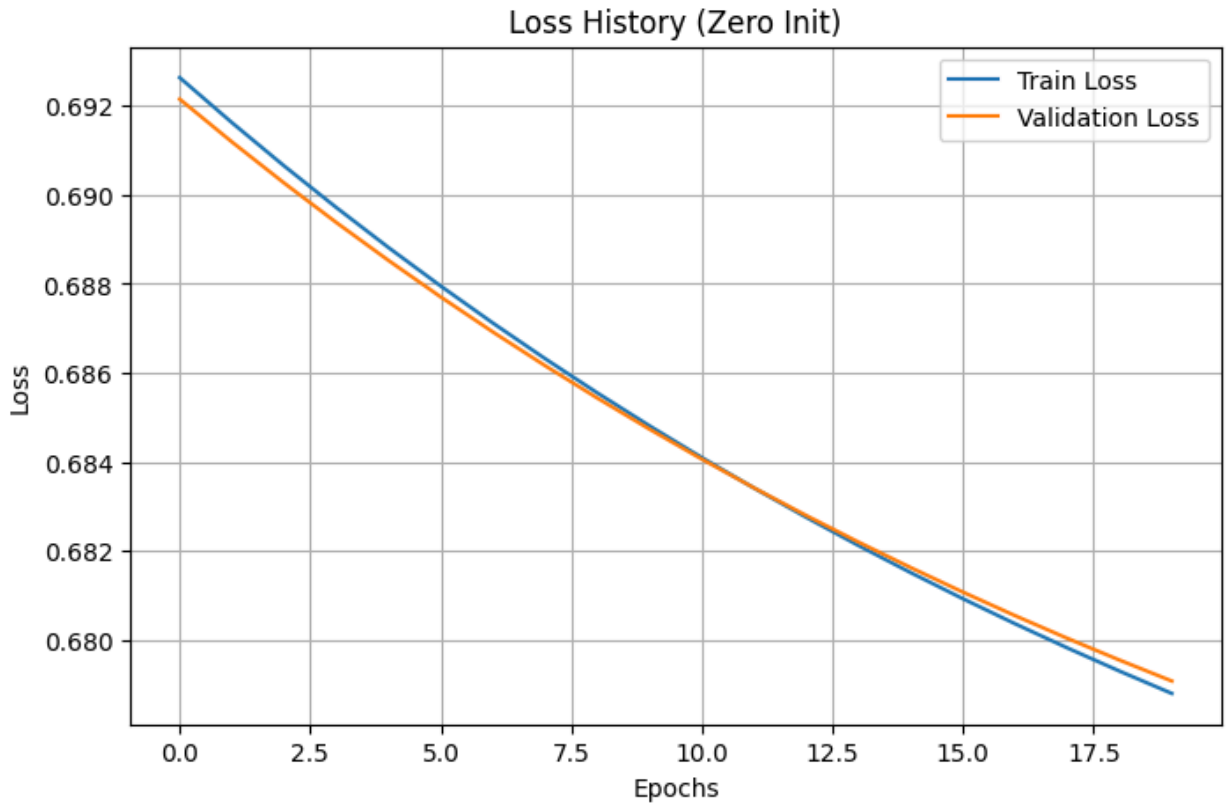

Layer 0 Weight
Layer 0 Bias
Layer 1 Weight
Layer 1 Bias

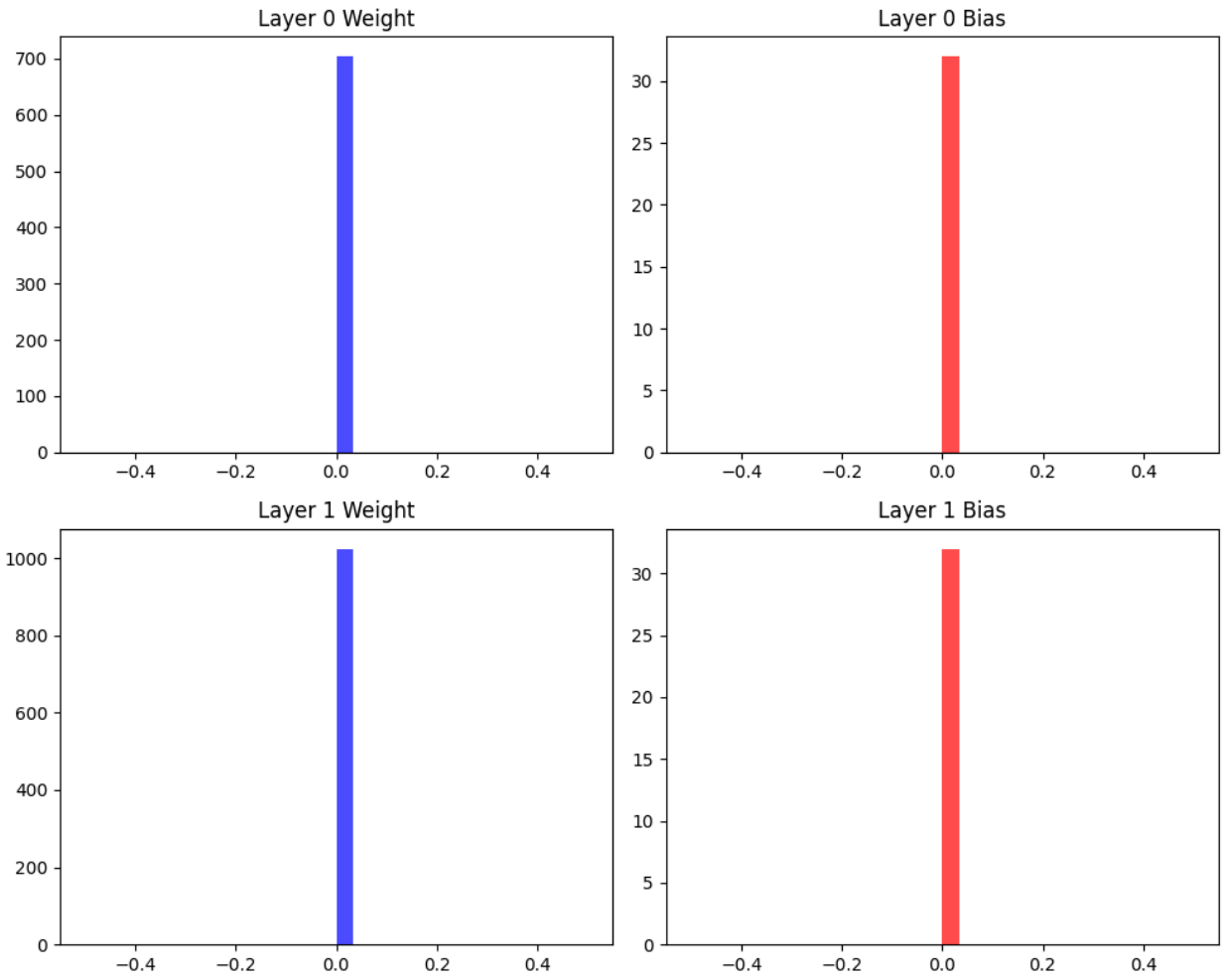


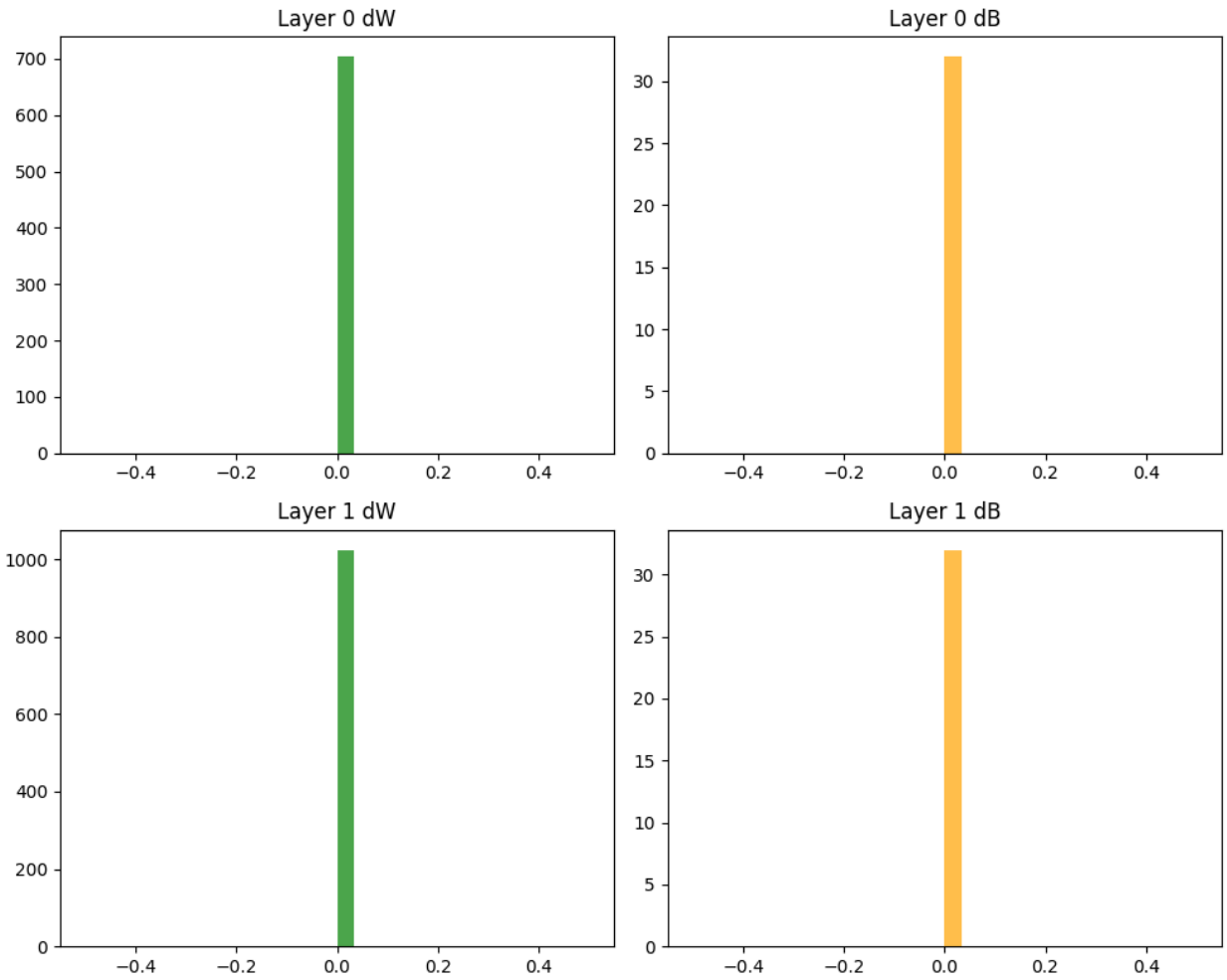
Pengurangan laju LR teramat rendah memberikan visualisasi graf melengkung lesu (garis menukik tipis dan memanjang persis menyerupai garis diagonal linear), tertinggal stagnan menyusut tipis saja dari nilai originalnya 1.4 dan cuma mendarat lemah menuju tingkat kegagalan loss 0.90. Gambar bentuk balok-balok histogram bobot gradiennya terlihat memiliki persebaran yang amat rapi namun nilai sumbunya terlalu sempit menandakan kurang besarnya rentang nilai baru yang diekplorasi.

2.2.4. Pengaruh inisialisasi bobot

a. Zero Init

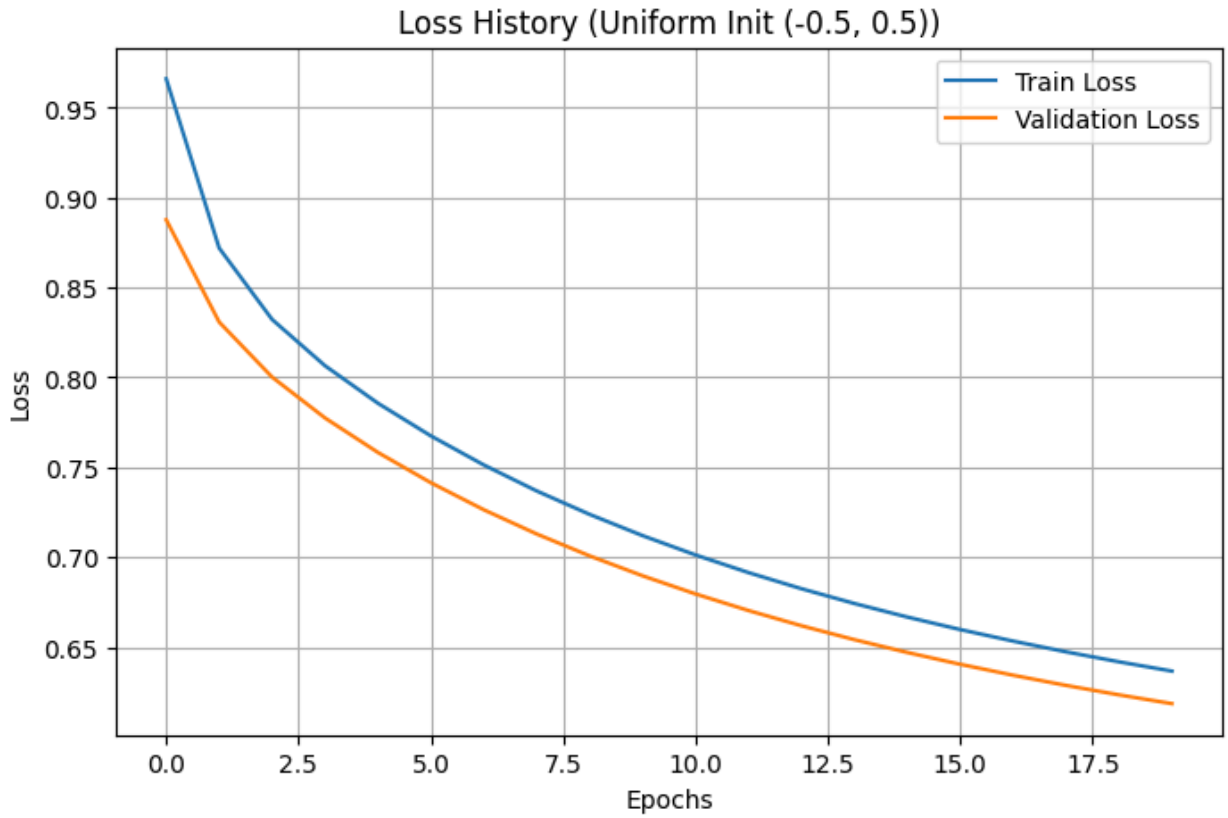


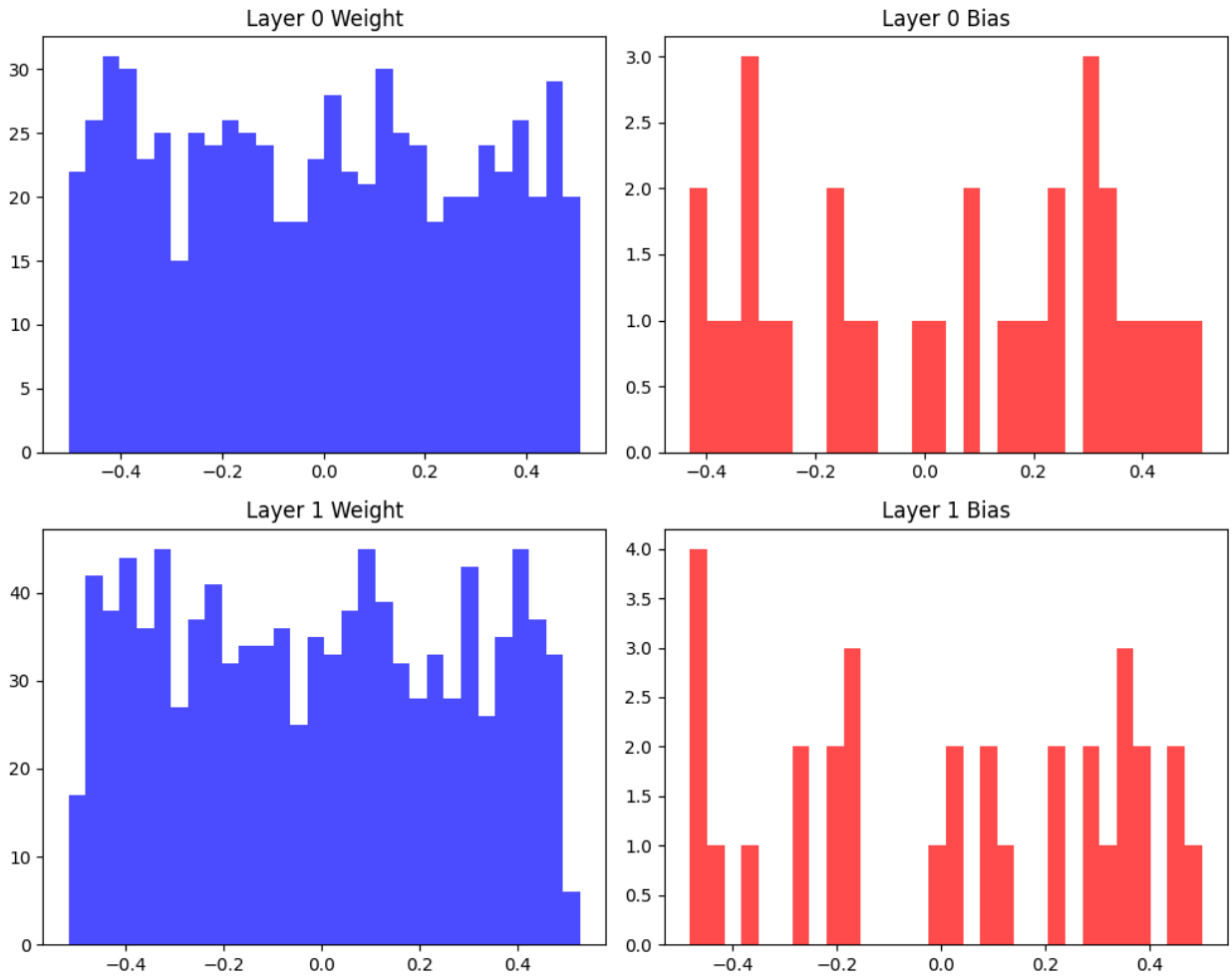

Layer 0 Weight
Layer 0 Bias
Layer 1 Weight
Layer 1 Bias

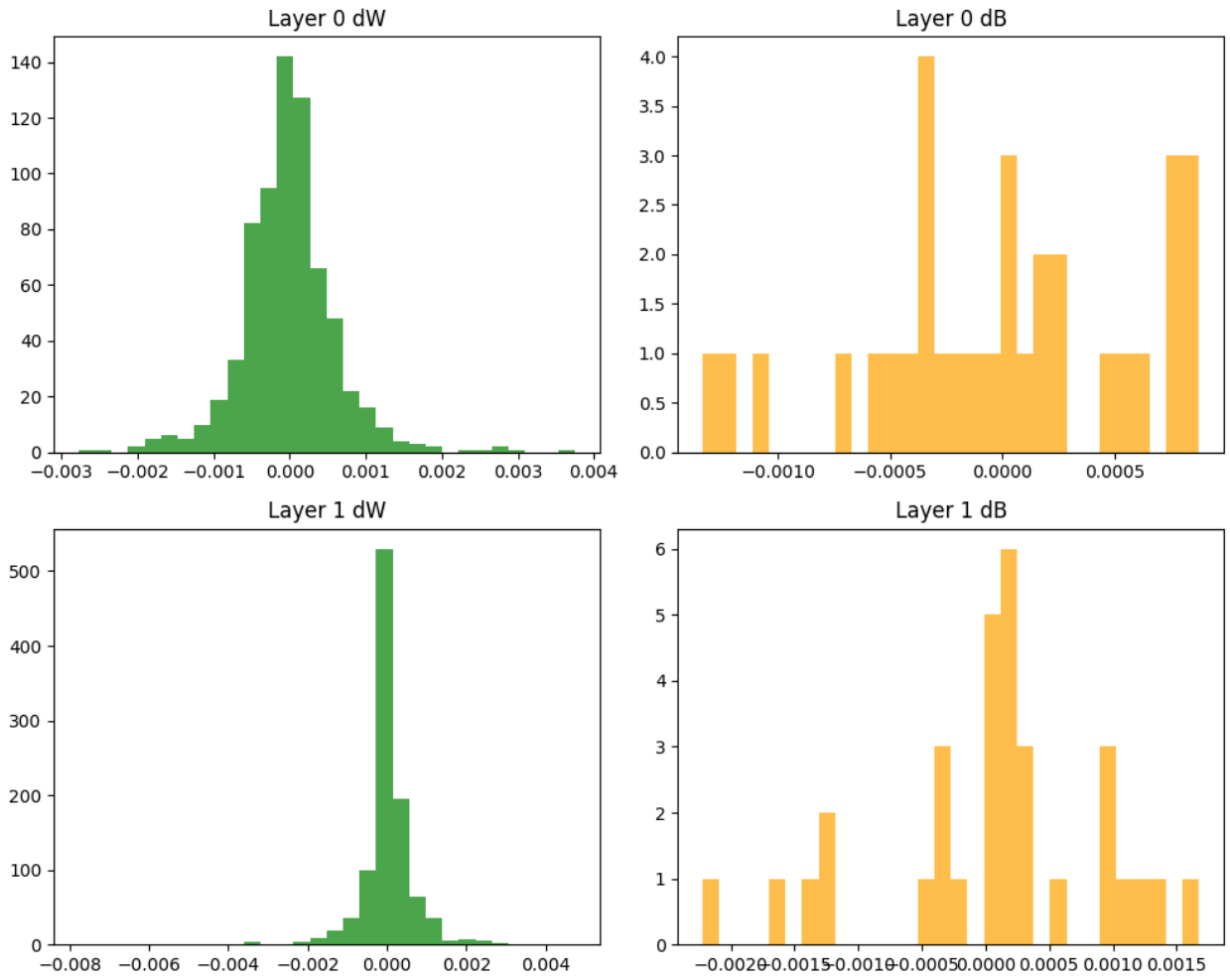


Kurva riwayat nilai Loss merentang selayaknya garis horizontal lurus tipis yang turun terlampau lambat hanya dalam margin luar biasa ketat di area sempit 0.692 menjadi 0.680. Seluruh distribusi batang histrogram Weight Layer 0 dan 1 murni tertumpuk statis menjadi satu kubus tiang menyendiri vertikal bernilai tunggal yakni absolut nol (0.0). Kegagalan pecah-simetri ini menjebak saraf sistem untuk tidak bisa belajar apa-apa dalam prosesnya.

b. Uniform Init (-0.5 - 0.5)

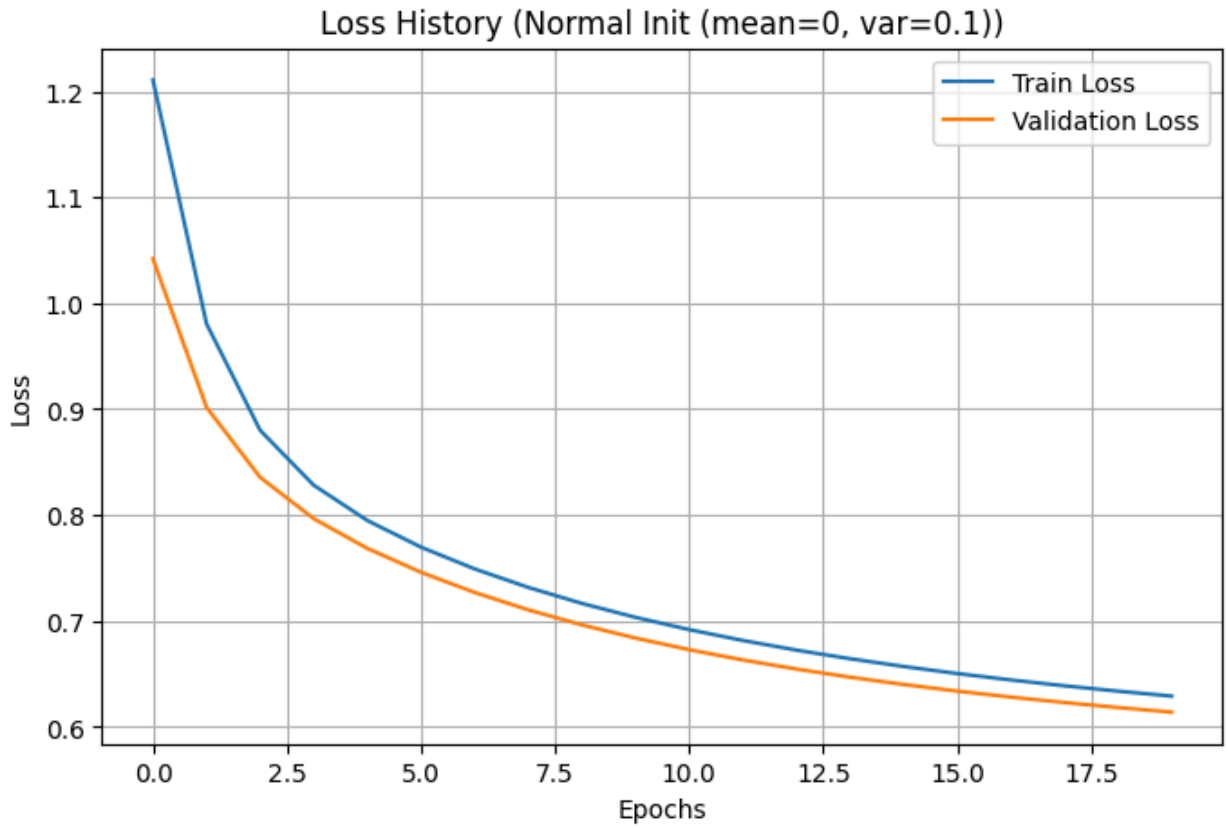


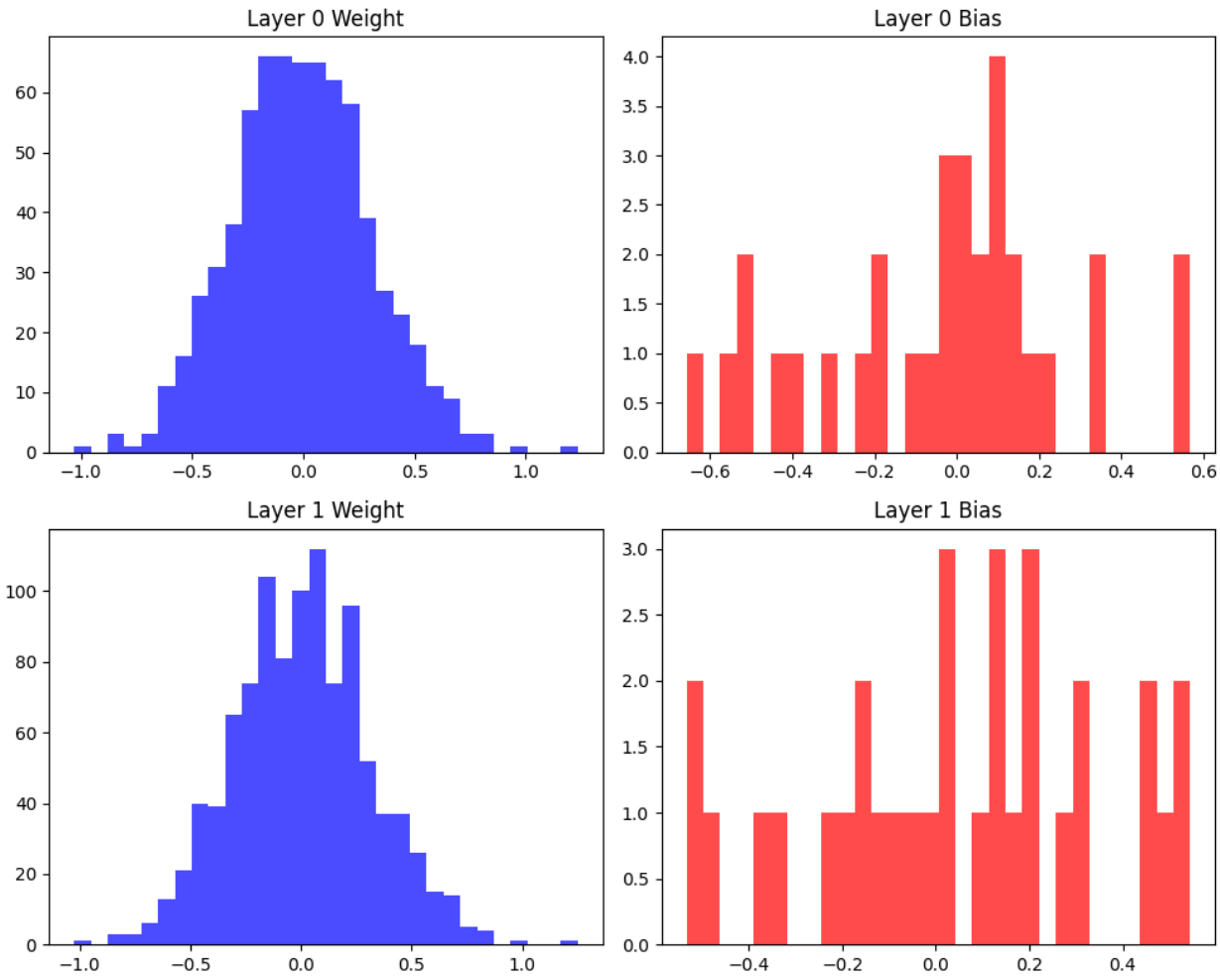

Layer 0 Weight
Layer 0 Bias
Layer 1 Weight
Layer 1 Bias

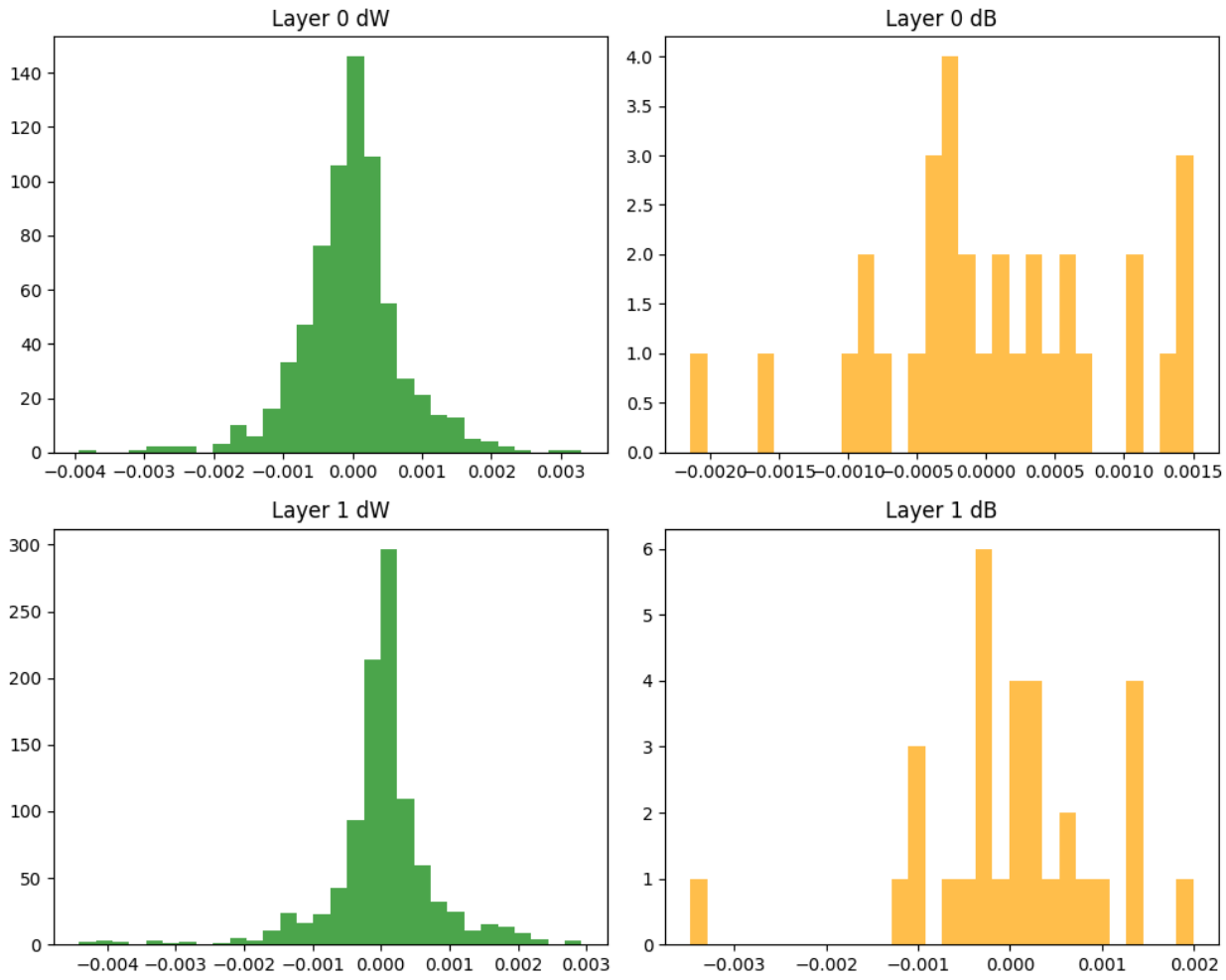


Model mendulang kemerosotan optimalitas loss curve menyusut dari nilai elevasi awal 0.95 menuju garis mendatar 0.64. Dibanding model sebaran bobot lain, graf histogram balok birunya mendapati karakteristik barak yang lebih pipih menyebar acak bersesakan menyesuaikan pola batas awal pendistribusian rataannya pada jangkauan batas minimal ke maksimal batas sumbunya.

c. Normal Init



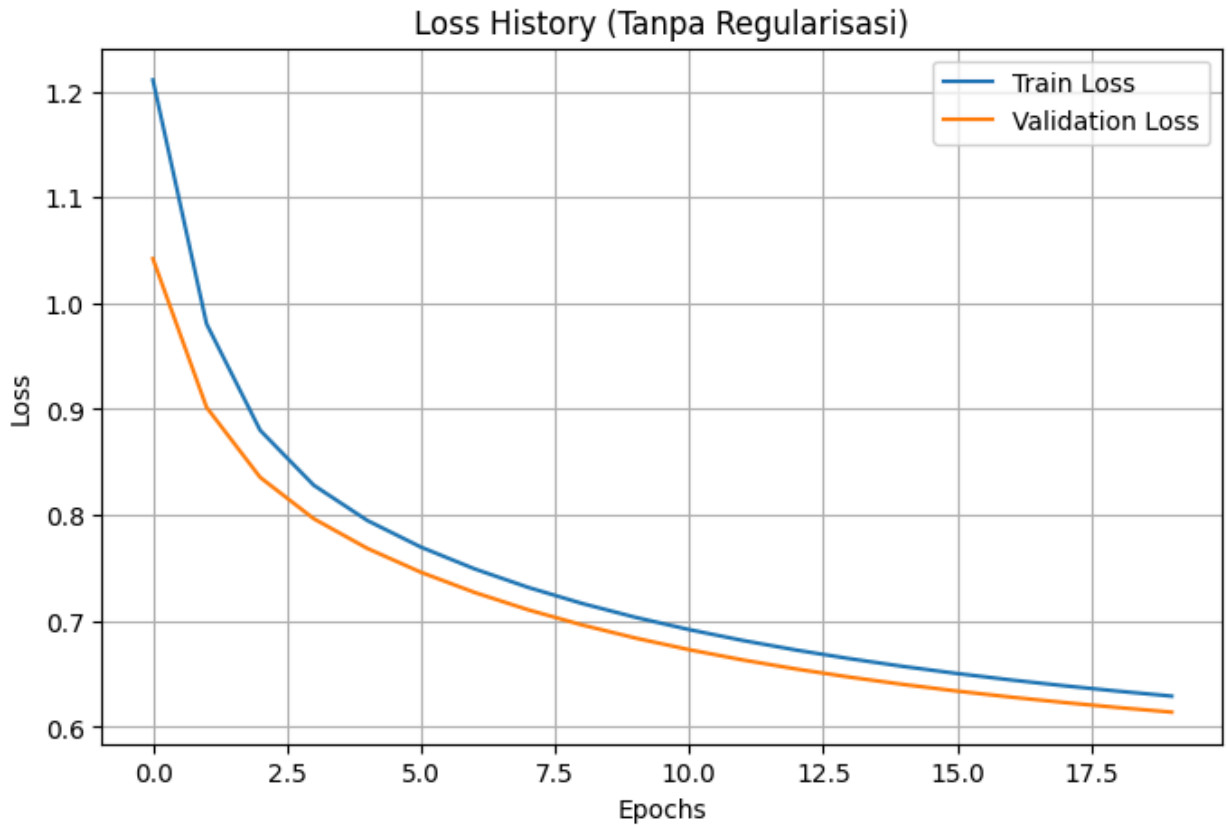

Layer 0 Weight
Layer 0 Bias
Layer 1 Weight
Layer 1 Bias

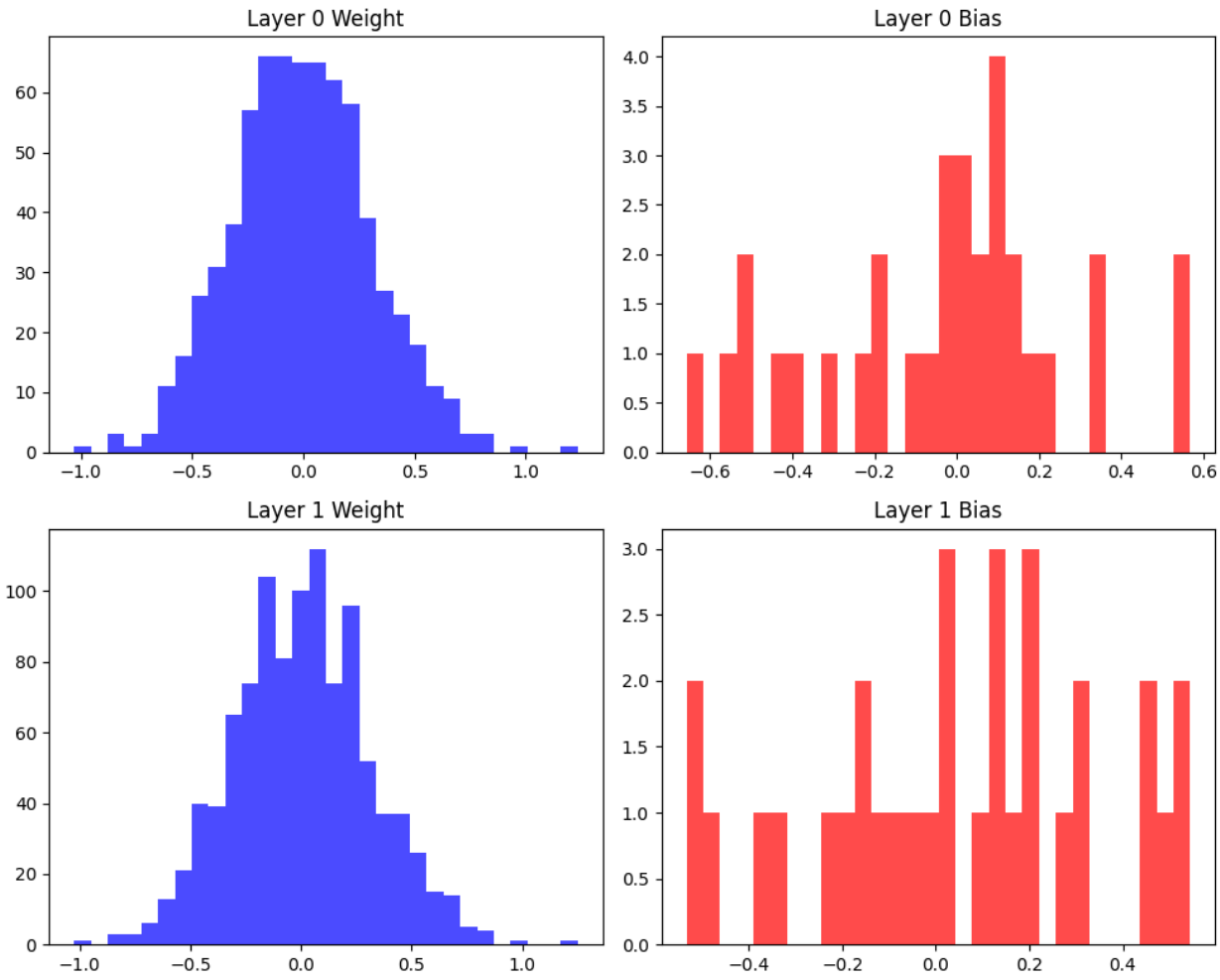


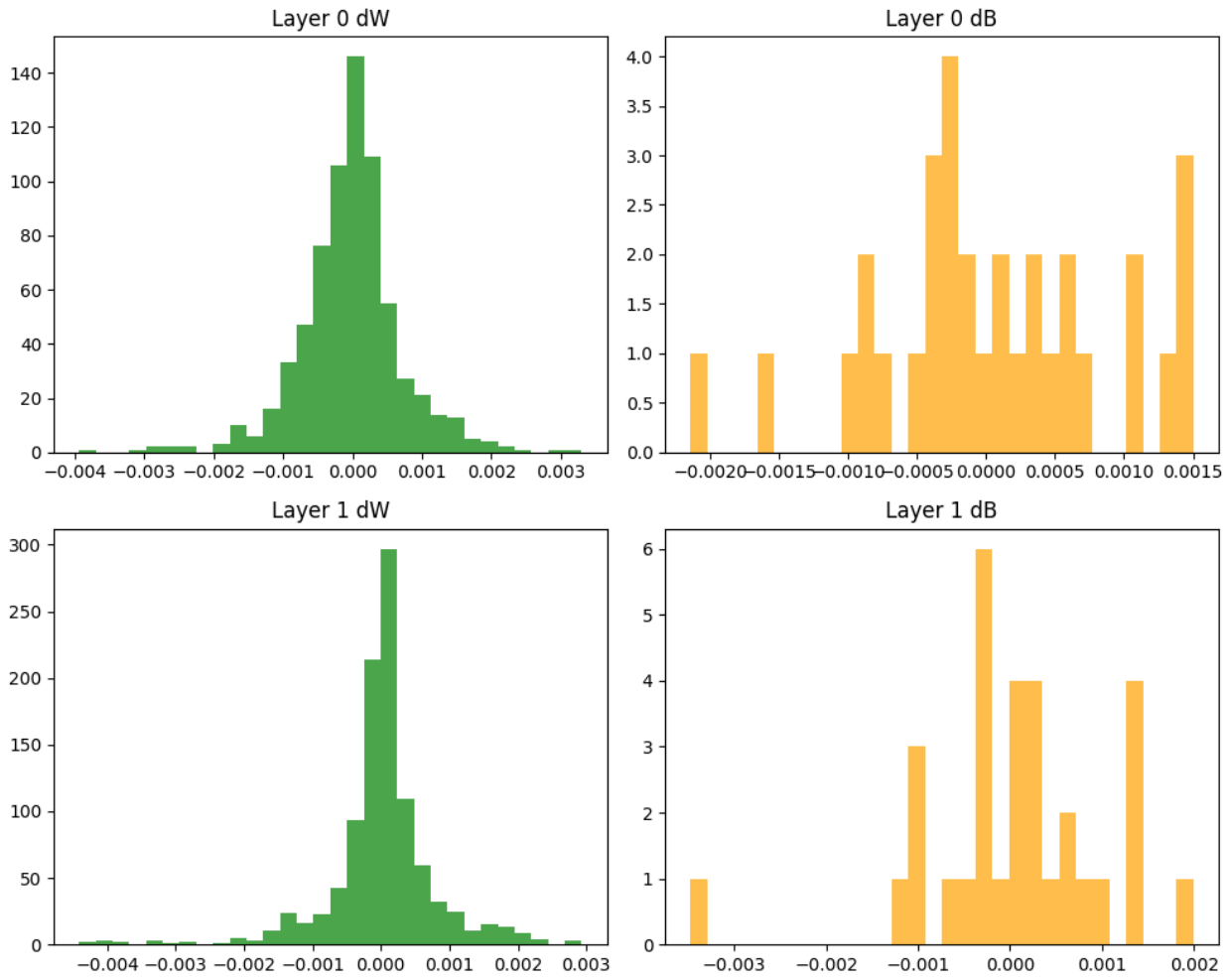
Garis evaluasi akurasi loss terpantau meniru ritme konvergensi normalisasi dari kisaran nilai awalan tinggi 1.2 mengerucut asimtotis ke ambang limit sekitar 0.63. Di sektor visual bar Weight Distribution, kita disajikan pola penggambaran normal bell-shaped alias lonceng simetris yang bertambah intens di kutub simetrik nol tengahnya mencerminkan hasil eksekusi turunan gradien wajar.

2.2.5. Pengaruh regularisasi

a. Tanpa Regularisasi

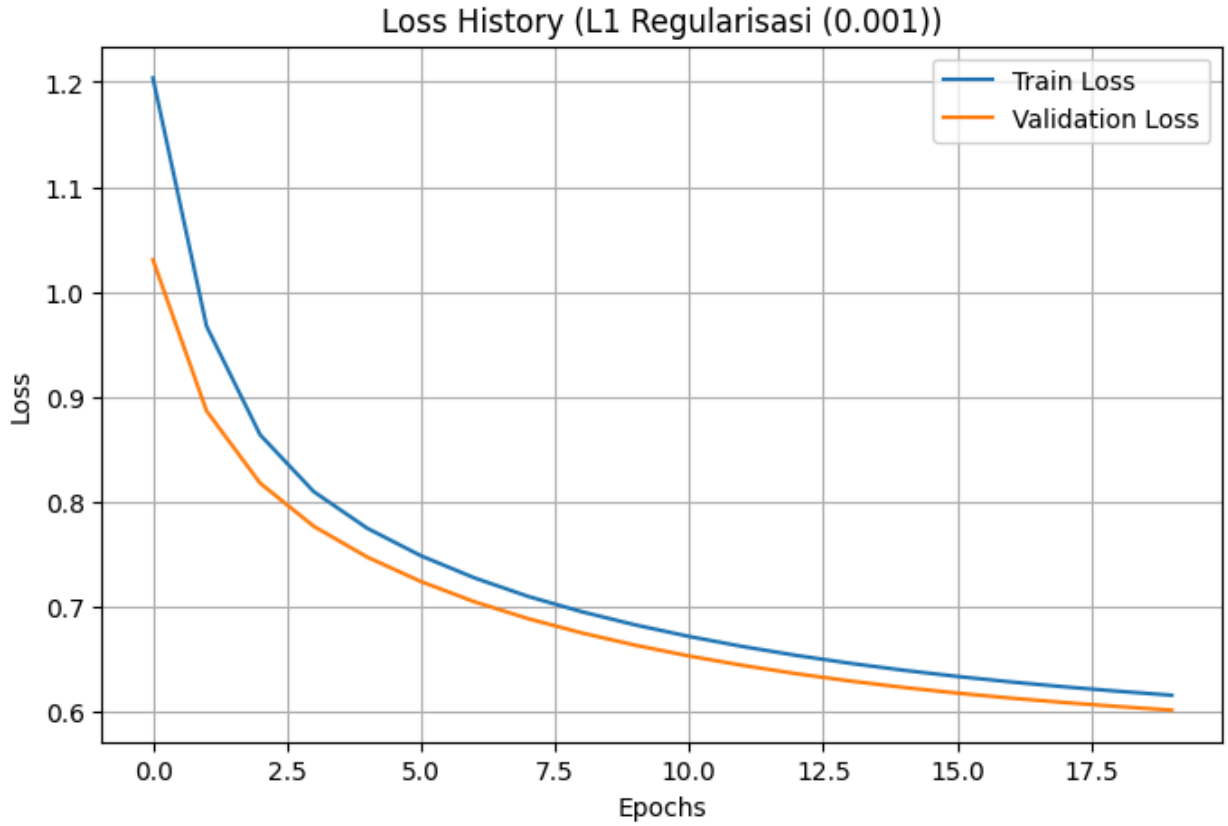


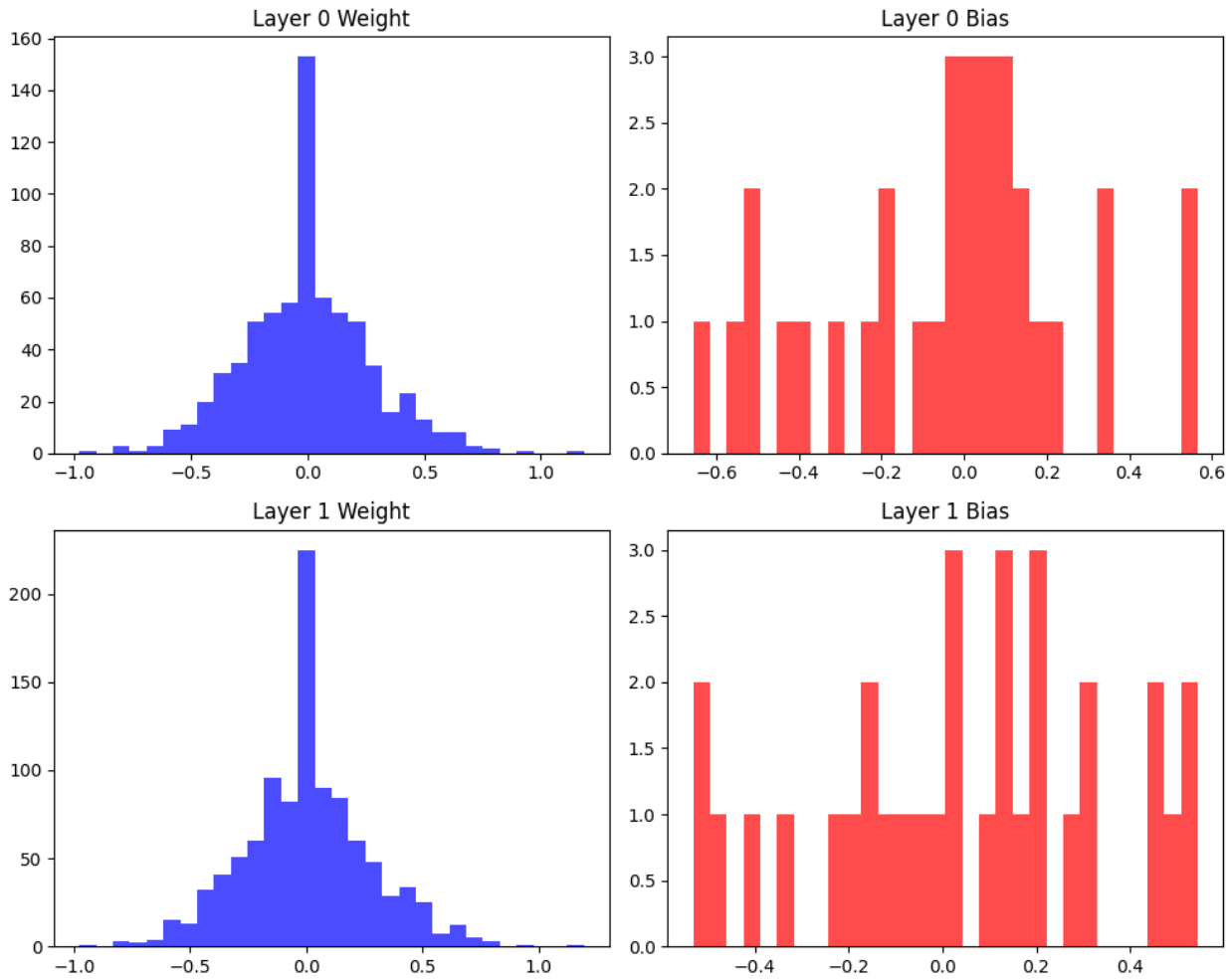

Layer 0 Weight
Layer 0 Bias
Layer 1 Weight
Layer 1 Bias

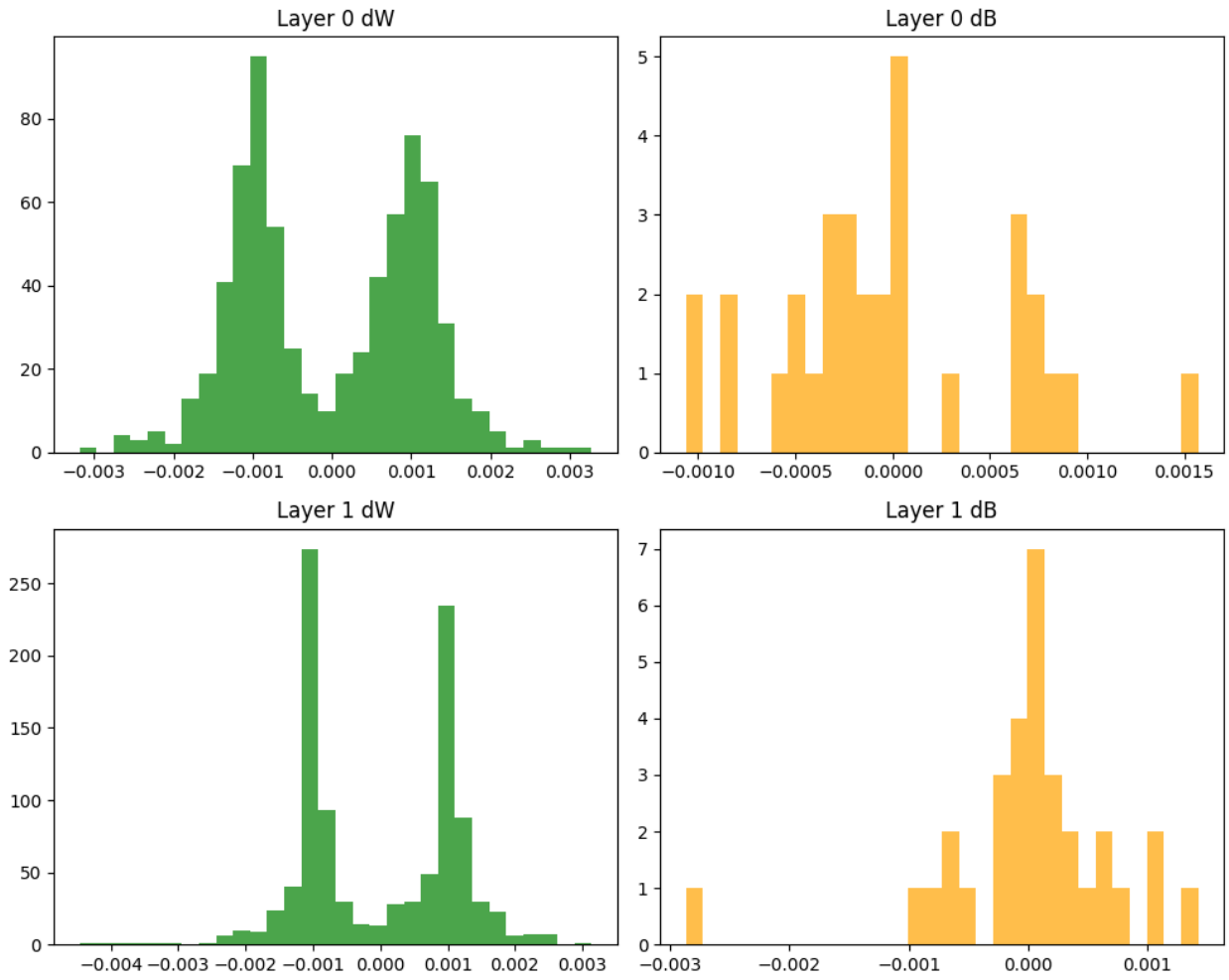


Perilaku graf model ini menghimpun turunan asimtotis biasa mengejar ke skala final validation loss 0.61, di mana jarak celah oranye serta rentang garis biru mengekor harmonis. Tidak tampak adanya cacat simpangan atas validasi (*no graphical overfitting curve*) serta grafik baloknya berbentuk kurva bukit biasa.

b. L1 Regularisasi (0.001)

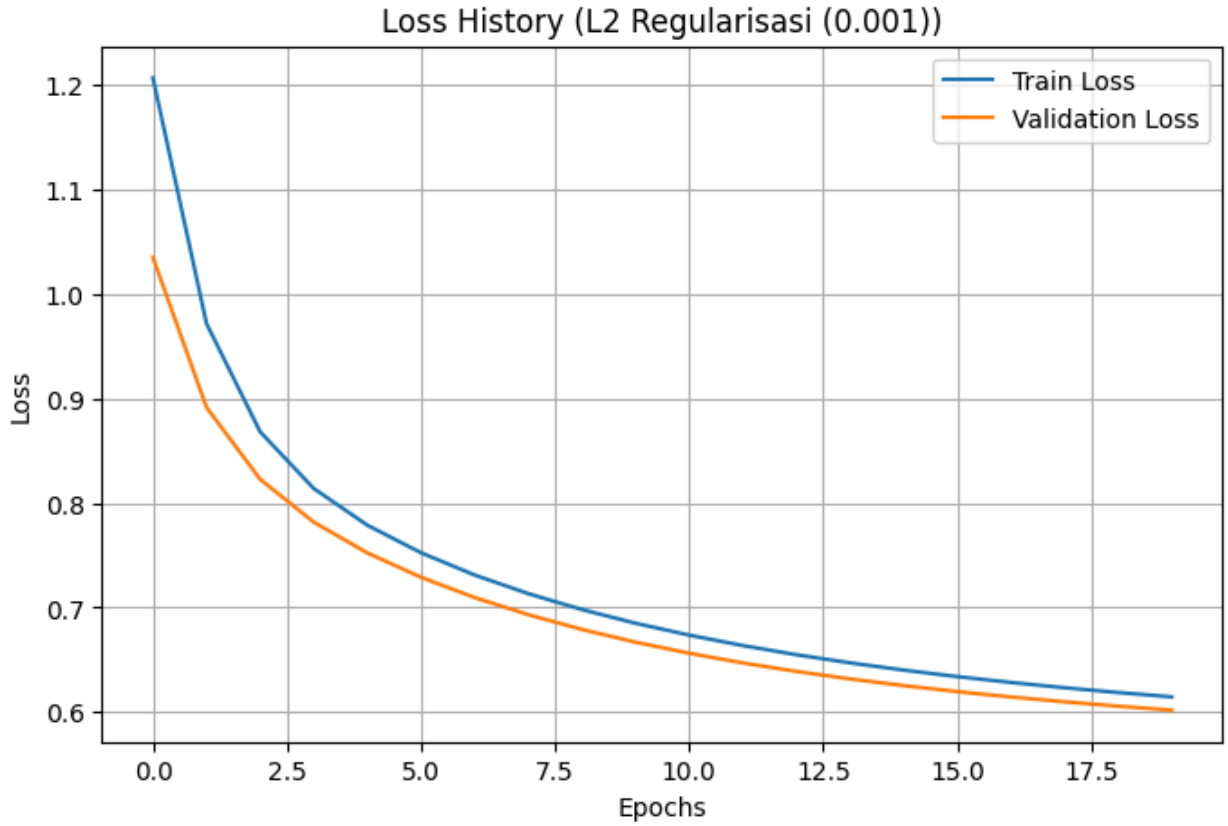


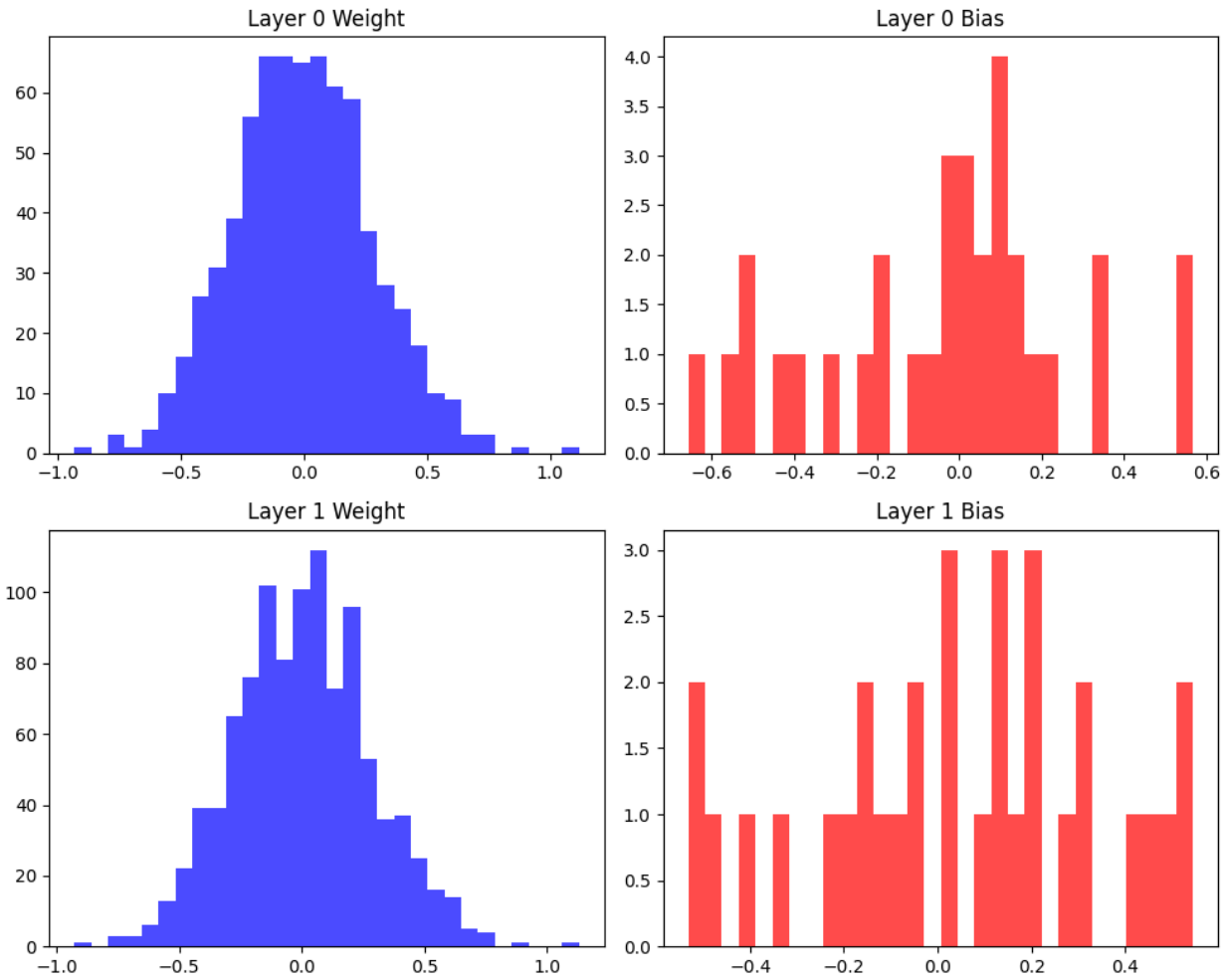

Layer 0 Weight
Layer 0 Bias
Layer 1 Weight
Layer 1 Bias

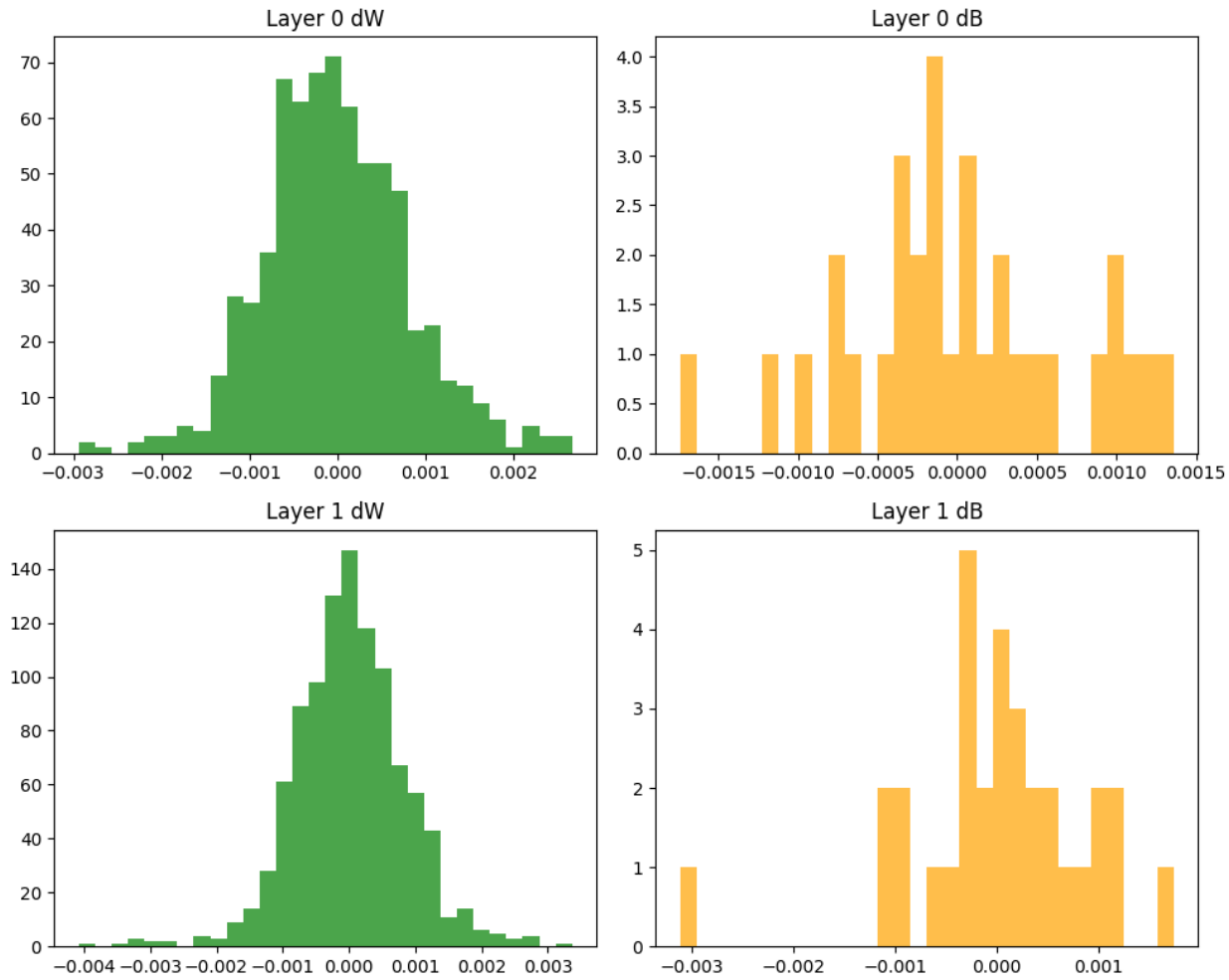


Skema grafiknya menyoroti kemiripan pola standar dengan ujung hasil tertambat konstan pada loss level 0.62. Hal memikat berada di bagian grafik Layer dW (histogram hijau), tidak lagi mengumpul sebagai 1 gunung puncak penengah—kini sebarannya menunjukkan formasi bimodal atau adanya robekan (gap) penurun pada puncak tengah 0.000 dengan baloknya yang terpecah ganda membumbung di sekitarnya. Hal unik dalam gambar ini merefleksikan kerja fitur L1 yang memaksa memangkas paksa persentase bobot kecil ke dimensi nol.

c. L2 Regularisasi (0.001)




Layer 0 Weight
Layer 0 Bias
Layer 1 Weight
Layer 1 Bias



L2 tidak mewujudkan efek penyusutan dua puncak spektrum layaknya graf dW milik L1. Seluruh balok di bagian grafik histogram biru maupun hijaunya kembali mendemonstrasikan kelancaran bukit Gaussian tanpa lubang tengah pemusatan nol. Graf line loss rates-nya meliuk mulus yang secara kasat mata persis mirip kembar identik dengan wujud grafik pada model Tanpa Regularisasi.

2.2.6. Perbandingan dengan library sklearn

Hasil akurasi FFNN yang telah dibuat adalah 0.6720. Sedangkan hasil akurasi sklearn MLPClassifier adalah 0.7440.

Kesimpulan dan Saran

Kesimpulan

Berdasarkan keseluruhan hasil pengujian yang telah dilakukan menggunakan arsitektur *Feedforward Neural Network* buatan sendiri, dapat ditarik beberapa kesimpulan utama:

1. Dari segi ukuran arsitektur (*depth dan width*), dataset parameter kelulusan mahasiswa yang dilatih rupanya tidak menuntut tingkat kompleksitas pencarian pola yang rumit. Penggunaan model dengan kedalaman dan lebar neuron yang minim (misalnya kedalaman 3 dan lebar 16) sudah cukup memadai untuk merengkuh tingkat konvergensi yang mulus dan bebas dari overfitting. Sebaliknya, memperbesar dimensi matriks hidden layer justru menunda kecepatan model untuk menyentuh titik minimum error dalam batas 20 epoch tersebut.
2. Dalam ranah pengujian fungsi aktivasi, pemakaian fungsi Linear secara tidak terduga memberikan lintasan evaluasi paling lancar dengan nilai batas loss terakhir yang paling rendah di antara yang lain. Fungsi ReLU dan Sigmoid menampilkan perilaku komputasi yang lazim, sementara pemakaian Tanh terbukti rentan terhadap instabilitas lonjakan loss (*spiking*) di awal iterasi sebelum akhirnya jaringan memulihkan diri.
3. Hyperparameter Tuning pada ukuran batasan rentang learning rate bertindak absolut bagi rentang konvergensi model. Laju setinggi 0.1 terlahir paling efisien mendorong model anjlok menuju area minimumnya secara cepat. Sebaliknya, laju yang ditekan terlalu kecil yakni sebesar 0.001 menyabotase performa; neuron diperbarui terlalu pelan (underfitting) hingga pelatihan kadaluarsa sebelum akurasi tercapai maksimal.
4. Kewajiban untuk merusak titik persilangan simetris jaringan lewat metode inisialisasi bobot standar seperti Uniform dan Normal adalah fakta krusial yang terbukti secara empiris. Inisialisasi secara konstan menggunakan metode Zero mutlak membuat neural network mati rasa, tak satupun perseptron mampu menarik perbedaan gradien sehingga model lumpuh dari mendeteksi pola pengetahuan apa pun.
5. Peran fungsi regularisasi (L1 maupun L2) pada dataset spesifik ini tidak berdampak signifikan pada grafik loss karena sifat alami varian model dasarnya

yang memang jarang menjumpai overfitting. Sungguhpun demikian, regularisasi L1 terbukti bertindak mengurai gradien distribusi yang terseparasi, menjalankan misinya untuk memecah kepadatan parameter.

6. Uji sandingan performa menunjukkan algoritma dasar buatan tangan manual masih harus mengakui supremasi akurasi milik fungsi pustaka tertanam library Scikit-Learn MLPClassifier (skor ~0.67 berbanding ~0.74). Selisih kepintaran tersebut terjadi lantaran model library komersial disokong oleh metode optimisasi internal tingkat lanjut, perhitungan konvergensi stokastik adaptif berlapis tinggi, serta heuristik matematis yang melampaui kemampuan implementasi pure gradient descent dasar.

Saran

Untuk meningkatkan performa kalkulasi matematis dan kualitas tebakan klasifikasi Feedforward Neural Network pada literasi pengembangan ke depannya, berikut adalah beberapa saran yang wajib dipertimbangkan:

1. Perpanjangan dan Optimasi Batas Epoch. Dialokasikan kuota putaran epoch yang lebih banyak melampaui 20 tahap, utamanya saat bereksperimen memakai arsitektur lebar dan nilai learning rate yang dipangkas kecil. Tujuannya adalah menggaransi jalinan parameter dapat konvergen mutlak menuju garis dasar asimtotis tanpa terpotong batas waktu secara prematur.
2. Pengembangan Optimizer Mutakhir. Sangat disarankan untuk beralih dan merakit algoritma dasar pembaru bobot (optimizer) berkelas adaptif seperti Adaptive Moment Estimation (Adam) agar bisa menyaingi kapabilitas library eksternal. Gradient descent polos yang digunakan sekarang memiliki respons terlalu stagnan dalam menyiasati pergerakan lembah gradien loss yang curam.
3. Fitur Early Stopping. Pemasangan baris algoritma instruksi pemberhentian awal secara otomatis (early stopping) amat direkomendasikan ketika parameter validation loss sudah dipindai berhenti membuat penurunan berarti setelah belasan epoch, semata demi penghematan daya eksekusi komputasi harian.
4. Eksplorasi Rekayasa Fitur Dataset. Karena pengetesan kurva Linear secara aneh terbukti sukses mengakali hasil kinerja fungsi kognisi setingkat ReLU, besar kemungkinan alur titik poin dataset orisinal ini masih tersebar secara dominan

linier. Ke depannya dapat dikerahkan campur tangan pembentukan fitur artifisial (feature engineering berformat polinomial) sebelum proses Input Layer, supaya potensi sejati dari rumusan aktivasi non-linear bisa terpacu lebih agresif lagi untuk mempersembahkan akurasi superior.

Pembagian tugas tiap anggota kelompok

NIM	Nama	Kontribusi
13523128	Andi Farhan Hidayat	Regularization, Visualization, Pemodelan, Pengujian, Laporan
13523145	Andri Nurdianto	Activation function, Loss function, EDA,, Laporan
13523146	Rafael Marchel D. W.	FFNN, Links, Laporan

Referensi

Deep Learning: Feed Forward Neural Networks (FFNNs)
https://medium.com/@b.terryjack/introduction-to-deep-learning-feed-forward-neural-networks-ffnns-a-k-a-c688d83a309d

What is Forward Propagation in Neural Networks
https://www.geeksforgeeks.org/deep-learning/what-is-forward-propagation-in-neural-networks/

Backpropagation in Neural Network
https://medium.com/@b.terryjack/introduction-to-deep-learning-feed-forward-neural-networks-ffnns-a-k-a-c688d83a309d

Lampiran

Penggunaan AI
https://docs.google.com/document/d/1h-iGIoEO32EkZzX3uvIJoETerZwLbuoSBOV_9ODkTkA/edit?usp=sharing

Repositori
https://github.com/drianto/KasihTauMama